S E L F - B A L A N C I N G R O B O T C O N T R O L

基于 STM32F4 的双轮自平衡机器人 LQR 与卡尔曼滤波实现

毕业设计完整方案 · 全套代码 · 技术栈 · 答辩面试手册

硬件平台：STM32F407VET6 + MPU6050 + TB6612FNG + 编码器电机

算法内核：二维卡尔曼姿态融合 · LQR 状态反馈最优控制

软件平台：STM32 HAL（Keil MDK / GCC）+ Python 上位机

适用场景：开题 / 中期检查 / 论文写作 / 答辩 / 求职面试

文档版本：V1.0　编制日期：2026 年 10 月

摘 要

本文档面向毕业设计《基于 STM32F4 的双轮自平衡机器人（LQR 最优控制 + 卡尔曼滤波）》，给出从被控对象建模、估计与控制算法推导、固件全套代码、上位机调参工具到答辩应答的完整工程方案。两轮自平衡车在力学上是一个本质不稳定的轮式倒立摆：以本文样车参数计算，1° 初始偏差经约 191ms 即发散至 10°，这决定了全链路必须以 5ms（200Hz）控制周期闭环。

系统采用 MPU6050 与正交编码器双传感源：加速度计长期无漂移但动态被运动加速度污染，陀螺仪短期精确但存在零偏漂移，二者缺陷恰好互补。本文按最小方差准则完整推导二维卡尔曼滤波器，状态取 [倾角, 陀螺零偏]，在线完成姿态融合与零偏自学习；随后用牛顿-欧拉法建立轮式倒立摆四状态空间模型（可测全状态，无需观测器），以变分法完整推导无限时间 LQR：哈密顿函数—协态方程—黎卡提代数方程—最优增益，并以实测机械参数离线求解 K。速度通道利用加减速必须先改变倾角的物理机理，采用速度 PI 输出目标角偏置的串级结构（先仰后动），并给出增广积分 LQI 备选方案。离散化采用精确 ZOH 并论证连续 K 直接可用的条件。

设计指标：静态平衡角度波动 ≤±1.0°、侧向冲击恢复时间 ≤2s、速度阶跃跟踪稳态误差 ≤0.02m/s、控制周期抖动 <1%。全部代码随文档交付，第 7 章给出可回填实测值的测试矩阵与典型故障排查手册，第 11 章提供 40 问答辩手册。

文档使用方法：开题阶段重点读第 1、2、10 章；动手前精读第 3 章参数测量方法；控制器设计跟随第 4 章推导与脚本复现；固件按第 5 章清单逐模块实现；调试标定按第 7 章顺序执行；答辩前用第 11 章自测。

关键词：两轮自平衡机器人；轮式倒立摆；LQR 最优控制；卡尔曼滤波；状态空间建模；MPU6050；速度闭环；STM32F4

目 录

第 1 章　项目定位与总体设计.....

第 1 章　项目定位与总体设计.....2

1.1 项目一句话定位与三种运行形态.....2

1.2 系统总体架构与控制回路.....3

1.3 与同类毕业设计的差异化对标.....3

1.4 量化设计指标.....4

1.5 为什么这套技术组合有竞争力.....4

第 2 章　理论基础与数学准备.....5

2.1 倒立摆为什么“本质不稳定”：线性化与时间尺度.....5

2.2 MPU6050 传感机理：两个“有缺陷的证人”.....6

2.2.1 寄存器级配置.....6

2.2.2 加速度计：测的是比力，不是角度.....7

2.2.3 陀螺仪：积分得角，漂移致命.....8

2.2.4 互补滤波：能用的起点，不是终点.....8

2.3 卡尔曼滤波：完整数学推导.....8

2.3.1 一维情形：从最小方差推出增益.....8

2.3.2 平衡车的二维模型：状态 [θ, b].....9

2.3.3 五条公式与逐项展开.....9

2.4 LQR 最优控制：完整推导.....11

2.4.1 问题定义.....11

2.4.2 变分法推导：哈密顿 → 协态 → Riccati.....12

2.4.3 鲁棒性保证（对比 PID 的硬货）.....14

2.5 先仰后动：速度控制的方向性原理.....14
第 3 章 硬件方案设计.....15
3.1 器件选型与关键参数.....15
3.2 关键电路设计与计算.....15
3.2.1 电机驱动电流预算.....15
3.2.2 PWM 频率与死区.....15
3.2.3 I2C 总线上拉电阻计算.....16
3.2.4 电源拓扑与去耦.....16
3.2.5 编码器接口.....17
3.3 误差预算.....17
3.4 机械与电机参数测量方法.....17
3.5 元器件清单（BOM）.....18
第 4 章 被控对象建模与控制器设计.....19
4.1 轮式倒立摆动力学建模（完整推导）.....19
4.2 状态空间实现与数值验证.....21
4.3 LQR 求解与离散化.....22
4.3.1 离线求解（清单 4-1）.....22
4.3.2 离散化：5ms 下连续 K 为什么可以直接用.....24
4.4 卡尔曼滤波器参数确定.....25
4.5 速度闭环结构设计.....26
4.5.1 串级结构：速度 PI 输出目标角偏置.....26
4.5.2 备选：增广积分 LQR（LQI）.....27
4.5.3 编码器速度量化：一个必须算的数字.....27

4.6 仿真验证.....27
4.7 与串级 PID 的系统对比.....32
第 5 章 STM32 固件设计（含完整代码）.....32
5.1 工程结构、开发环境与外设分配.....32
5.2 控制周期与时序预算.....33
5.3 全局参数配置.....33
5.4 MPU6050 驱动.....40
5.5 卡尔曼姿态解算.....50
5.6 编码器测速.....55
5.7 控制器：LQR + 速度环.....59
5.8 电机驱动与死区补偿.....64
5.9 遥测帧协议.....71
5.10 主程序：启动 gating 与故障安全.....75
5.11 编译、烧录与常见坑.....85
第 6 章 上位机与调参工具（含完整代码）.....86
6.1 开发环境与依赖.....86
6.2 串口协议解析（状态机）.....86
6.3 实时曲线与在线调参面板.....92
6.4 无车联调：先打通上位机，再碰实车.....101
第 7 章 测试验证与结果分析.....102
7.1 测试平台与仪器.....102
7.2 测试用例与记录表.....102
7.3 标定与整定流程（按序执行，一步没绿不进下一步）.....102

7.4 典型故障排查手册 103
第 8 章　技术栈总表与简历改写 104
8.1 技术栈分层全景 104
8.2 一分钟项目介绍（面试/答辩开场通用） 104
8.3 简历项目经历改写（对照版） 105
第 9 章　开发计划与里程碑（10 周） 105
第 10 章　论文与开题写作映射 106
10.1 论文章节结构建议（六章制） 106
10.2 图表清单 106
10.3 参考文献基础列表 107
第 11 章　面试与答辩手册（40 问） 108
11.1 传感器与滤波类（10 问） 108
11.2 建模与动力学类（8 问） 112
11.3 LQR 与控制类（10 问） 115
11.4 嵌入式实现类（7 问） 118
11.5 工程素养与压力问题（5 问） 121
附录 A 代码文件清单与自检结果 123
附录 B 开源参考与甄别建议 124

说明：本目录由域代码生成。文档编辑后请在目录上右键选择“更新域→更新整个目录”以刷新页码。

第 1 章　项目定位与总体设计

1.1 项目一句话定位与三种运行形态

本项目是一台从传感器误差机理到最优控制理论全部闭环的双轮自平衡机器人：以 STM32F407 为实时控制节点，MPU6050 姿态原始数据经二维卡尔曼滤波融合出倾角与角速度，编码器给出轮速，被控对象按轮式倒立摆建模并离线求解 LQR 最优增益，5ms 定时中断内完成“读—滤—算—写”全闭环；速度通道利用“加减速必须先改变倾角”的物理机理做串级补偿。它不是调出来的 demo，而是每个参数都能回答“为什么是这个值”的控制系统。

同一套硬件支撑三种运行形态，按调试阶段切换：

- 形态 A（边缘自治直立）：STM32 本地完成 I2C 采集、卡尔曼解算、LQR 与速度环，断开上位机也可独立站立与行驶，是验收的基本形态；

- 形态 B（上位机遥测调参）：STM32 以 921600bps 上传 100Hz 遥测帧（角度、角速度、车速、控制量、卡尔曼零偏），Python 上位机用 pyqtgraph 实时绘制并支持在线修改速度环参数，是第 7 章整定工作的主战场；

- 形态 C（遥控行驶）：上位机/蓝牙下发目标速度 v_ref 与转向差速量，车体完成速度跟踪与差速转向，用于轨迹类演示。

> **定位要点**　答辩先讲边界：姿态估计解决“车以为自己歪多少”，LQR 解决“知道自己歪了之后手脚怎么动”，速度环解决“想去哪”。三层边界讲清楚，再展开任何一层细节，评委的跟随成本最低。

1.2 系统总体架构与控制回路

系统分为感知层、边缘层、执行层三级，如图 1-1 所示。感知层由 MPU6050（I2C@400kHz 突发读 14 字节）与两路正交编码器（定时器编码器模式 4 倍频）组成，分别提供姿态与轮速；边缘层以 5ms 定时器中断为脊柱：卡尔曼滤波完成姿态融合与零偏自学习，LQR 状态反馈 u = -K·x 与速度 PI 在同一中断内合成控制量，经死区补偿与限幅后写 PWM；执行层由 TB6612FNG 驱动双直流减速电机，车体作为轮式倒立摆把控制量“物理地”反馈回传感器。

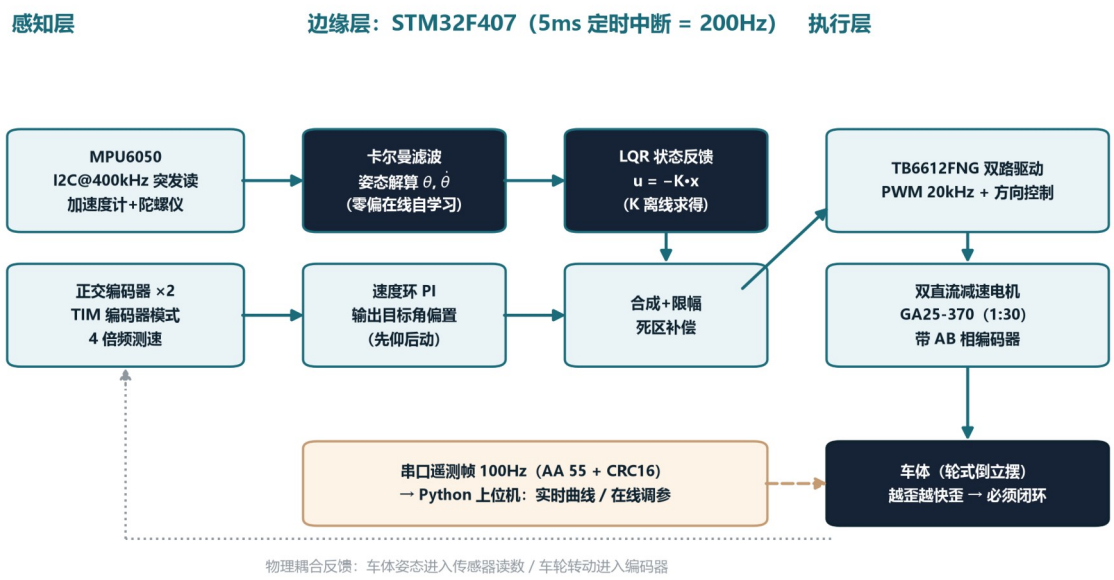


图 1-1 系统总体架构（感知层—边缘层—执行层，200Hz 实时闭环）

控制回路的一句话版本：卡尔曼告诉控制器“车现在歪多少、歪多快”，编码器告诉它“轮子跑多快”，LQR 按离线算好的四个权重把这两个信息合成一个力矩指令，电机执行后车体的物理响应又回到传感器——整个链条每 5ms 走一遍。为什么必须这么快，第 2.1 节给出定量答案。

1.3 与同类毕业设计的差异化对标

市面上以 STM32 为平台的自平衡车毕设大多停留在“DMP 直读角度 + 串级 PID 试凑”的层面。表 1-1 从七个维度列出本项目与常见做法的差别，每一行都是答辩时可以直接展开的技术点。

表 1-1 与常见同类毕设的对比

维度	常见同类毕设	本项目的做法
姿态获取	MPU6050 DMP 硬件直读欧拉角	寄存器级驱动 + 自研二维卡尔曼滤波，零偏在线估计，全部参数物理可解释
姿态融合依据	互补滤波系数 0.98 拍脑袋	最小方差准则推导增益 $K=P^{-}/(P^{-}+R)$，R 由实测噪声方差确定
被控对象建模	不建模，把车当黑盒	牛顿-欧拉法建立轮式倒立摆四状态模型，机械参数实测（摆线法/两工况法）

维度	常见同类毕设	本项目的做法
控制器	直立环 PD + 速度环 PI 逐环试凑	LQR 一次解出全部增益，Q/R 权重整定，附鲁棒裕度保证
速度环	速度误差直接叠加 PWM	利用先仰后动机理输出目标角偏置，与直立环物理机理自洽
离散化	默认采样就行	精确 ZOH 离散 + 论证 5ms 下连续 K 直接可用的误差量级（约 0.3%）
调试方法	改参数-看现象循环	仿真先行（第 4 章脚本）+ 三步标定仪式（符号/中值/死区）+ 上位机遥测归因

1.4 量化设计指标

设计指标来自倒立摆时间尺度、传感器噪声特性与执行器能力三条线的交集，见表 1-2。表中【实测值】在完成第 7 章测试流程后填写，论文与答辩均以填写后的表格为准。

表 1-2 量化设计指标与验收方法

指标项	设计目标	依据 / 验收方法	实测值
控制周期 / 抖动	5ms，jitter < 1%	2.1 节时间尺度推导；GPIO 翻转 + 示波器实测	【待填】
姿态解算输出率	200Hz，滞后 ≤20ms	卡尔曼 R 整定权衡，见 2.3 节	【待填】
静态平衡波动	θ 波动 ≤±1.0°	上位机记录 60s 静态数据统计峰值	【待填】
抗冲击恢复	侧推 10° 摆角恢复 ≤2s	第 7 章 T2 用例，遥测曲线量取	【待填】
速度阶跃跟踪	0→0.3m/s 稳态误差 ≤0.02m/s	T3 用例，编码器读数	【待填】
上电自站立成功率	≥90%（20 次统计）	启动 gating 策略，见 5.10 节	【待填】
采样与控制丢拍	0	遥测帧 SEQ 连续性统计	【待填】
连续运行时间	≥30min 无重启	IWDG 看门狗 + 长跑测试	【待填】

1.5 为什么这套技术组合有竞争力

这套项目的价值在于把四个知识面拧成一条真实闭环：传感器与估计理论（比力、方差、最小方差估计）、经典力学与建模（牛顿-欧拉、线性化、能控性）、现代控制（变分法、Riccati 方程、状态反馈、鲁棒裕度）、嵌入式实时系统（定时器、I2C、编码器接口、PWM、看门狗）。单看任何一个点都是课程内容，但能把四个点做成一条“每个参数都有出处、每次失稳都能归因”的闭环，就超过了绝大多数只做其中一两个点的同类项目，也正是控制/嵌入式岗位面试官最愿意追问的完整链条。

第 2 章　理论基础与数学准备

本章把全项目用到的三块理论一次推干净：倒立摆为什么不稳定（2.1）、两个姿态传感器各自的机理与缺陷（2.2）、以及两大算法的完整推导——卡尔曼滤波（2.3）与 LQR（2.4）。所有推导保留中间步骤，所有类比只做助记。

2.1 倒立摆为什么“本质不稳定”：线性化与时间尺度

车体绕轮轴转动（轴为非惯性系，惯性力项下一章正式推导），小角度下的转动方程可以写为：

$$I_b \cdot \ddot{\theta} = M_p \cdot g \cdot L \cdot \theta - M_p \cdot L \cdot \ddot{x}$$

先看最坏情形 $\ddot{x}=0$（轮轴不动）：$\ddot{\theta} = (M_p \cdot g \cdot L / I_b) \cdot \theta$。角度的二阶导与角度成正比、系数为正——这正是“越歪越快歪”的数学形式，与单摆（系数为负、来回摆）恰好相反。以样车参数 $M_p=1.5\text{kg}$、$L=0.1\text{m}$、$I_b=0.02\text{kg}\cdot\text{m}^2$ 代入，放大系数 $M_pgL/I_b \approx 73.5\ \text{s}^{-2}$，含车轮耦合后的精确值为 245.8（见 4.2 节），对应开环不稳定极点：

$$s = \pm\sqrt{245.8} \approx \pm 15.7\ \text{rad}^{-1}$$

按 $\theta(t) = \theta_0 \cdot \cosh(15.7t)$ 计算，从 1° 涨到 10° 需要 $t = \text{arcosh}(10)/15.7 \approx 0.19\text{s}$，见图 2-1。这 191ms 就是全部软硬件的时间预算：传感器、滤波、计算、执行加起来必须显著快于它。取 10 倍以上裕量，控制周期定为 5ms（200Hz）——每 40 个控制拍内车才会倒到 10°，每一拍都有“看-想-动”的机会。这就是为什么平衡车项目用主循环 + delay 跑控制几乎必然失败：周期不稳等于给正反馈系统随机加延迟，抖动直接进姿态。

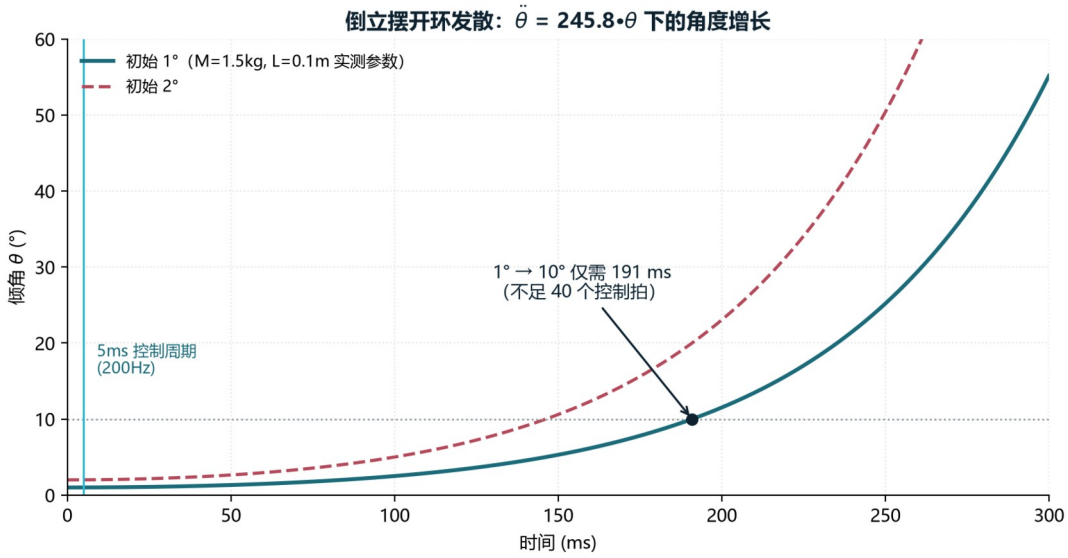


图 2-1 开环发散曲线与控制周期选择依据（$\ddot{\theta} = 245.8 \cdot \theta$，实测参数）

答辩高频点 被问“为什么选 200Hz”时，不要答“大家都是 5ms”。先报不稳定极点 15.7rad/s，再报 1°到 10° 的 191ms，最后说控制周期取不稳定时间常数的 1/40——这条推理链比结论本身值钱。

2.2 MPU6050 传感机理：两个“有缺陷的证人”

2.2.1 寄存器级配置

I2C 从机地址 0x68（AD0 接地）。上电初始化顺序与理由：

表 2-1 MPU6050 关键寄存器配置

寄存器	写入值	作用与理由
0x6B PWR_MGMT_1	0x01	退出睡眠；时钟源选 PLL（锁定陀螺仪），比内部 RC 振荡器噪声低
0x1A CONFIG	0x03	内部 DLPF=42Hz：先压电机机械振动（数百 Hz），又不过度引入相位滞后
0x1B GYRO_CONFIG	0x18	±2000dps 量程 → 灵敏度 32768/2000 = 16.4 LSB/(°/s)，防 fast fall 饱和
0x1C ACCEL_CONFIG	0x00	±2g 量程 → 16384 LSB/g，姿态角分辨率最优
0x75 WHO_AM_I	读 0x68	上电自检：读不到说明接线/上拉/时序问题，先修通信再谈控制

数据读取：从 0x3B 起一次性突发读 14 字节（加速度 XYZ、温度、陀螺 XYZ，各 16 位大端）。一次事务保证 14 字节属于同一采样时刻，分开读会在两次事务之间跨越采样更新，造成“加速度是 t 时刻、陀螺是 t+1 时刻”的撕裂数据。

2.2.2 加速度计：测的是比力，不是角度

加速度计感受的是比力（支撑反力），静止时数值上等于反重力方向。设 MPU 的 x 轴指向车头、z 轴向上，车前倾 α（前倾为正）。把向上的比力 g 投影到机体系：

$$ax = -g\cdot\sin\alpha,\quad az = g\cdot\cos\alpha \quad\Longrightarrow\quad \alpha = \operatorname{atan2}(-ax, az)$$

注意 atan2 里那个负号来自投影方向，安装方向一变它就变——这正是第 7.3 节符号自检仪式的第一项。静态分辨率很高：±2g 量程下 1 LSB 折合 0.0035°。但动态下完全不可信：车以加速度 $\ddot{x}$ 前进时 x 轴真实读数变为 $ax = -g\cdot\sin\alpha - \ddot{x}$，多出的 $-\ddot{x}$ 会被 atan2 当成倾角。定量：0.98 m/s²（很温和的加速）造成 $\arctan(0.98/9.8) \approx 5.7°$ 的假倾角——比真实姿态扰动还大。

> **大白话类比** 加速度计像一个被电机吵得发抖的证人：安静时说的话可信，一有动静就胡说，但说得再久也不会漂移。陀螺仪是另一个证人：反应极快、短期极准，但每说一句话就悄悄偏离事实一点（零偏），说久了完全跑偏，而且永远不会自己回来。法官该信谁、信几分，就是滤波问题。

2.2.3 陀螺仪：积分得角，漂移致命

陀螺仪直接输出角速度 ω（°/s），积分得角：$\theta(t) = \theta(0) + \int(\omega_raw - b)dt$，离散形式 $\theta \mathrel{+}= (gyro - b)\cdot dt$ 就是后面卡尔曼的预测步。零偏 b 永远不会是零：MPU6050 出厂零偏规格 ±3°/s，温漂另算。b = 0.5°/s 时一分钟漂 30°，完全不可用。所以上电必须静止采 400 个样本求均值作为 b（5.3 节），且卡尔曼还要在线继续学它（2.3 节）。

2.2.4 互补滤波：能用的起点，不是终点

最简单的融合是一阶互补：$angle = k\cdot(angle + gyro\cdot dt) + (1-k)\cdot acc_angle$。陀螺通道构成高通（短期信自己），加速度计通道构成低通（长期信观测），k=0.98 附近

效果尚可。但 k 是拍的：振动大的车和安静的车最优 k 不同，而且它学不出零偏。把“该信谁多少”从拍脑袋变成按噪声统计特性严格推导，就是卡尔曼滤波。

2.3 卡尔曼滤波：完整数学推导

2.3.1 一维情形：从最小方差推出增益

设角度预测值 x^-（来自陀螺积分）方差为 P^-，观测值 z（加速度计反正切角）噪声方差为 R。最优估计取残差修正形式 $\hat{x} = x^- + K(z - x^-)$，估计误差方差为：

$$P = (1-K)^2 \cdot P^- + K^2 \cdot R$$

对 K 求导取零：$dP/dK = -2(1-K)P^- + 2KR = 0$，解出：

$$K = P^- / (P^- + R)$$

这就是卡尔曼增益的全部秘密：预测方差大（积分漂了）则 $K \to 1$ 多信观测，观测噪声大（车在抖）则 $K \to 0$ 多信预测。信任比例由方差之比唯一决定，不是拍的。

> **大白话类比** K 就是法官分配给两个证人的话筒音量：谁刚才那段说得越没底（方差大）话筒音量调得越低。而“没底”的程度，P^- 和 R 两个数字严格算出来。

2.3.2 平衡车的二维模型：状态 [θ, b]

一维版学不出零偏。实际取二维状态 $x = [\theta, b]^T$（θ 倾角、b 陀螺零偏，设为零偏随机游走），把陀螺读数 g 作为已知输入：

三个噪声参数的物理意义：Q_angle 是陀螺积分模型每步积累的误差方差（模型失配），Q_bias 是零偏随机游走强度，R 是加速度计角度观测方差（可实测）。

2.3.3 五条公式与逐项展开

标准卡尔曼递推为预测两式 + 更新三式。代入本问题的 F、B、H 逐项展开（这就是写进 STM32 的全部内容）：

预测① $\hat{x}^- = F \cdot \hat{x} + B \cdot u$：　$\theta^- = \theta + (g - b) \cdot dt$，　$b^- = b$

预测② $P^- = F \cdot P \cdot F^T + Q$：　$p11^- = p11 - dt \cdot (p12 + p21) + dt^2 \cdot p22 + Q_angle$

$p12^- = p12 - dt \cdot p22$，　$p21^- = p21 - dt \cdot p22$，　$p22^- = p22 + Q_bias$

更新③ $K = P^{-}H^{T}(H{\cdot}P^{-}{\cdot}H^{T} + R)^{-1}$：　$K_1 = p11^{-}/(p11^{-} + r)$，　$K_2 = p21^{-}/(p11^{-} + r)$

更新④ $\hat{x} = \hat{x}^{-} + K(z - H{\cdot}\hat{x}^{-})$：　$\theta = \theta^{-} + K_1{\cdot}(z - \theta^{-})$，　$b = b^{-} + K_2{\cdot}(z - \theta^{-})$

更新⑤ $P = (I - K{\cdot}H){\cdot}P^{-}$：　$p11 = (1-K_1)p11^{-}$，　$p12 = (1-K_1)p12^{-}$，

$p21 = p21^{-} - K_2{\cdot}p11^{-}$，　$p22 = p22^{-} - K_2{\cdot}p12^{-}$

对照 2.3.1 一维推导：K_1 就是“角度该信观测多少”（分子分母与 $K = P^{-}/(P^{-}+R)$ 同构）；K_2 是“观测残差有多少该记到零偏头上”——残差若持续同号，说明陀螺在漂，K_2 把它慢慢吸进 b。所以卡尔曼不仅滤了角度，还顺带在线辨识了零偏，这是一维加权平均做不到的。图 2-2 是按本节公式仿真的效果：陀螺积分 20 秒漂 16°，加速度计噪声 ±1.6°，卡尔曼输出全程贴合真值。

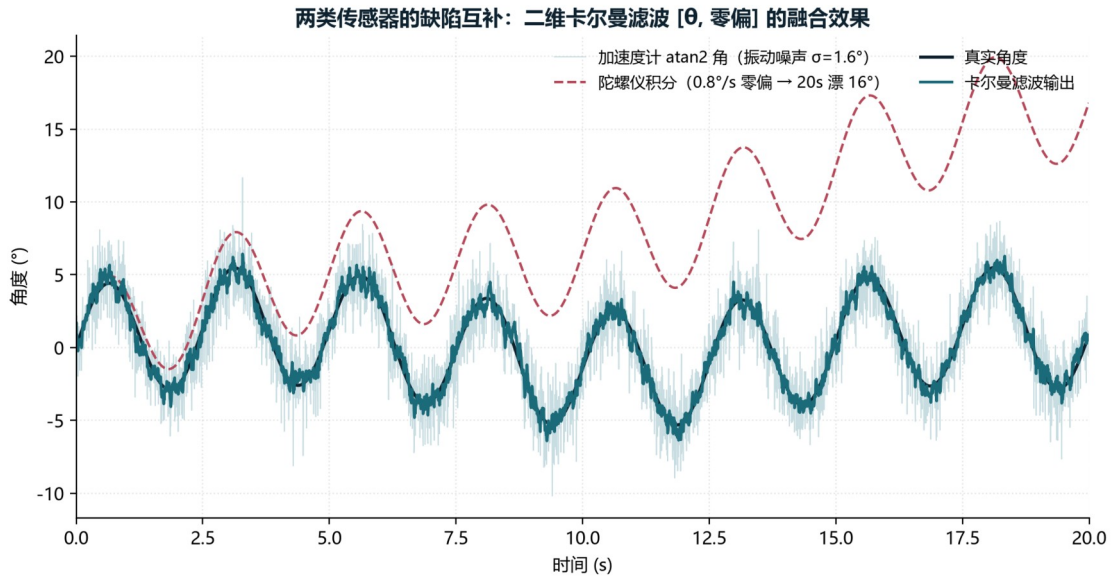


图 2-2 二维卡尔曼滤波融合效果（参数 Q_angle=0.001, Q_bias=0.003, R=0.03，单位度²）

与互补滤波的关系：P 在几个周期内收敛到稳态值，K 变成常数，此时递推形式上退化为互补滤波。所以互补滤波 = 稳态化、简化版的卡尔曼；卡尔曼 = 自动算出最优“0.98”、且能同时估计零偏的互补滤波。二者在 STM32 上成本都极低（每周期约 20 次浮点乘加），没有理由不用卡尔曼。

> **答辩高频点**　为什么状态取 [θ, b] 而不是 [θ, θ̇]？θ̇ 陀螺仪直接测得到，不需要估计；零偏才是真正未知且慢变的量。若取 [θ, θ̇] 并把陀螺当输入，两个状态全由同一输入驱动、观测只含 θ，θ̇ 通道不可观，滤波器是退化的。这一问答对，估计理论这一关就过了。

2.4 LQR 最优控制：完整推导

2.4.1 问题定义

给定线性系统 $\dot{x} = Ax + Bu$，找线性状态反馈 $u = -Kx$，使无限时间性能指标最小：

$$J = \tfrac{1}{2}\int_0^{\infty} [\, x^T \cdot Q \cdot x + u^T \cdot R \cdot u \,]\, dt, \quad Q \succeq 0,\ R \succ 0$$

> **大白话类比** Q 是每条状态的“罚款单价”：歪 1° 罚多少、跑快 1m/s 罚多少；R 是“油钱单价”：控制量每用 1 单位罚多少。LQR 求的就是总罚款加油钱最省的驾驶方式。调参 = 改价目表，改完理论替你算出全部 K，这是它和 PID 逐个旋钮试凑的本质区别。

2.4.2 变分法推导：哈密顿 → 协态 → Riccati

构造哈密顿函数（即时代价 + 协态 λ 与动力学的内积）：

$$H = \tfrac{1}{2}x^T Q x + \tfrac{1}{2}u^T R u + \lambda^T (Ax + Bu)$$

最优解的三个必要条件——状态方程、协态方程、极值条件：

$$① \ \dot{x} = \partial H / \partial \lambda = Ax + Bu$$

$$② \ \dot{\lambda} = -\partial H / \partial x = -Qx - A^T \lambda$$

$$③ \ 0 = \partial H / \partial u = Ru + B^T \lambda \quad \Longrightarrow \quad u = -R^{-1} B^T \lambda$$

条件③说明最优控制是协态的线性函数。猜测 $\lambda = P \cdot x$（P 为待定对称矩阵——这一步就是“最优反馈必然线性”的来源）：

$$u = -R^{-1} B^T P \cdot x = -K \cdot x, \quad K = R^{-1} B^T P$$

把 u 代回动力学闭环 $\dot{x} = (A - BK)x$，求出 $\dot{\lambda}$ 的另一表达式，与条件②对比：

$$\dot{\lambda} = \dot{P}x + P \cdot \dot{x} = \dot{P}x + P(A - BK)x$$

令其等于 $-Qx - A^T P x$，对任意 x 成立：

$$-\dot{P} = A^T P + PA - PBR^{-1}B^T P + Q$$

无限时间稳态时 $\dot{P} \to 0$，得代数 Riccati 方程（ARE）：

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

解出对称正定解 P（MATLAB lqr / Octave lqr 内部用 Schur 方法），增益随之确定：$K = R^{-1}B^T P$。最优代价 $J^* = \tfrac{1}{2}x_0^T P x_0$——P 的物理意义是“从当前状态出发的总罚款余额”。

2.4.3 鲁棒性保证（对比 PID 的硬货）

对全状态反馈 LQR，回路增益 $K(sI-A)^{-1}B$ 满足经典下界（Kalman 回程差结果）：每个回路至少 60° 相位裕度、至少 6dB（2 倍）增益裕度、无穷增益缩减裕度。含义：电机出力比标称少一半、传感器有中等相位滞后，理论上不会直接失稳。前提是“全状态反馈”成立——本项目 θ、θ̇ 来自卡尔曼，v、s 来自编码器，四个状态全可测，不需要设计观测器；也正因为如此，卡尔曼的质量直接决定这条保证兑现多少：滤波滞后吃掉的相位，就是从这 60° 里扣的。

2.5 先仰后动：速度控制的方向性原理

轮子对车的水平推力只能通过车体倾斜产生：重力沿车体纵向的分量近似 $\ddot{x} \approx g\cdot\sin(\theta_bias) \approx g\cdot\theta_bias$。要获得减速度 a，必须先向后仰 $\arcsin(a/g) \approx a/g$。这就是平衡车与普通调速电机的本质区别：速度控制的执行机构是姿态本身。定量：要把 0.5m/s 的超速在 1 秒内消掉（$a=0.5m/s^2$），需要后仰 $\arcsin(0.5/9.8) \approx 2.9°$。由此推出速度环的正确出口：不是直接给力矩，而是给直立环一个目标角偏置 $\theta_bias = -k\cdot(v - v_ref)$，加 I 项消静差。完整结构设计见 4.5 节。

> **大白话类比** 人急刹车身体后仰、起跑身体前倾——平衡车的加减速必须先改变姿态。车速超调时“先猛冲一下再倒回来”不是 bug，是物理：它必须先仰角减速。

第 3 章　硬件方案设计

3.1 器件选型与关键参数

表 3-1 核心器件选型

器件	型号/参数	选型理由
主控	STM32F407VET6 核心板，Cortex-M4F 168MHz	硬件 FPU（单精度浮点乘加 1 周期），200Hz 下卡尔曼+LQR 合计不足 0.5% CPU；TIM 编码器接口×2、I2C、高级定时器 PWM 齐全
姿态传感	MPU6050 模块（I2C，±2g / ±2000dps）	六轴成本低、资料全；内建 42Hz DLPF 先压机械振动；毕业设计工作量聚焦在算法而非传感器驱动移植
驱动	TB6612FNG 双路电机驱动	MOSFET 低内阻（持续 1.2A/峰值 3.2A），压降远小于 L298；20kHz PWM 无啸叫
电机	GA25-370 减速比 1:30，带 AB 相霍尔编码器（13 线）	额定 12V 下轮缘速度约 0.8m/s 覆盖平衡车工况；编码器提供速度闭环必需的反馈
电源	2S 锂电池 7.4V + LDO 3.3V	电机直供电池避免 LDO 压差发热；逻辑电由 LDO 独立供电并加去耦
车架	两层亚克力/铝板，重心高约 0.1m	重心高度 L 决定不稳定极点（见 2.1），不宜过矮，否则控制带宽要求反而苛刻

3.2 关键电路设计与计算

3.2.1 电机驱动电流预算

样车整备质量约 1.8kg，轮半径 0.03m，平地加速 2m/s² 所需轮缘推力约 3.6N，双轮合计力矩约 0.11N·m；GA25-370 减速后额定转矩 0.15N·m、堵转转矩约 0.6N·m，力矩储备 5 倍以上，满足抗冲击需求。电侧核算：电机额定电流约 0.3A、堵转约 1.6A，双路同时堵转 3.2A 触及 TB6612 峰值极限，固件上把归一化输出限幅在 95% 并加装 5A 自恢复保险丝兜底。平衡车正常运行很少进入持续堵转，但“最坏情况已知且行为可控”是可靠性设计的底线。

3.2.2 PWM 频率与死区

PWM 取 20kHz（TIM1 ARR=4199，84MHz/20kHz−1）。低于 1kHz 会进入人耳听觉段产生啸叫，且电流纹波大、电机发热；20kHz 远超人耳并使电流纹波 <5%。TB6612 内置死区保护直通，无需软件干预。

3.2.3 I2C 总线上拉电阻计算

MPU6050 挂在 I2C1（PB8/PB9），400kHz 快速模式。上升时间由上拉电阻与总线电容决定：$t_r \approx 0.8473 \cdot Rp \cdot Cb$。快速模式要求 $t_r \le 300ns$，两条线 + 模块走线总电容按 20pF 估算，得 $Rp \le 300ns/(0.8473 \times 20pF) \approx 17.7k\Omega$；同时驱动端下拉电流上限 3mA 要求 $Rp \ge 3.3V/3mA \approx 1.1k\Omega$。取常用值 4.7kΩ，上升时间约 80ns，居中且稳妥。I2C 布线尽量短于 10cm 并远离电机功率线。

3.2.4 电源拓扑与去耦

- 功率路径：电池 → 5A 保险丝 → TB6612 VM；电机回流不经过逻辑地路径；

- 逻辑路径：电池 → LDO 3.3V → MCU/MPU6050，LDO 输入输出各 10μF 钽 + 100nF 陶瓷；

- MPU6050 模块就近 100nF 去耦；I2C 上拉接 3.3V 干净域；

- ADC 采样电池电压（分压 1/11）用于输出归一化（见 7.4 故障 8）。

3.2.5 编码器接口

AB 相输出接 TIM3/TIM4 的 CH1/CH2，定时器编码器模式 Count 4 倍频自动计数，CPU 零开销。13 线×4 倍频×减速比 30 = 输出轴每转 1560 计数。速度由 5ms 内计数差分得到，量化噪声分析见 4.5 节。

3.3 误差预算

把姿态估计的总误差拆到每一级，是判断系统是否可信的标准方法，也是答辩时证明你理解系统的最好材料。表 3-2 按器件典型规格给出误差预算。

表 3-2 姿态与速度误差预算（按平方和开方合成）

误差源	典型量级	说明 / 对策
陀螺零偏校准残差	0.05°/s	上电 400 样本均值；残余由卡尔曼 K_2 通道在线吸收
陀螺温漂	0.02~0.05°/s	慢变，被当作零偏随机游走建模（Q_bias）
陀螺白噪声	0.05°/s @200Hz	积分后成随机游走，靠加速度计观测拉回
加速度计振动噪声	σ≈1.5°（atan2 角）	机械振动主导；DLPF 42Hz + 卡尔曼 R 加权压制
运动加速度假倾角	0.1g ≈ 5.7°	加速度计动态不可信的根源，靠陀螺通道主导短期
安装轴向/中值误差	0.3~1°	标定消除；残余 0.5° 折合 0.086m/s² 等效加速度（7.3 节）
编码器速度量化	0.024m/s @5ms	4.5 节定量；一阶低通压制，代价是相位滞后
RSS 合成（静态 θ）	≈±0.4°	满足表 1-2 的 ±1.0° 指标，中值标定后可再收敛

3.4 机械与电机参数测量方法

LQR 的 K 是从模型算出来的，模型参数必须实测而不是抄数据手册。四项关键参数的测法（半天工作量，回报极大）：

- 质心高度 L：吊线法。把整车吊起两次（不同悬挂点），铅垂线交点即质心，量取到轮轴的距离；

- 车体绕轴惯量 I_b：摆线法。拆除车轮让车体绕轮轴自由微摆，秒表测 20 个周期取平均 T_p，按 $I_b = M_p \cdot g \cdot L \cdot T_p^2/(4\pi^2)$ 计算；样车 $T_p \approx 1.04s$，得 $I_b \approx 0.020 kg \cdot m^2$；

- 电机反电动势常数 Ke 与内阻 R_m：两工况法。稳态下 $U = Ke \cdot \omega + I \cdot R_m$，给两个不同占空比测 (U, ω, I) 解二元一次方程；SI 单位制下 Kt = Ke，测一个参数等于拿到两个；

- 陀螺/加速度计零偏：上电静止各采 400 样本取均值，写入驱动（5.3 节）。

> **答辩高频点** 被问“你的模型参数哪来的”，报实测方法与数值，再补一句：摆线法本质是用重力恢复力标定转动惯量，与 $I_b=M_p \cdot L^2$ 的细杆近似相比把误差从约 30% 压到 5% 以内，这直接决定 LQR 的 K 准不准。

3.5 元器件清单（BOM）

表 3-3 BOM 清单（核心部分）

序号	器件/物料	规格	数量
1	STM32F407VET6 核心板	含 SWD 调试口、晶振、LDO	1
2	MPU6050 模块	GY-521，I2C	1
3	直流减速电机	GA25-370，1:30，AB 相编码器 13 线	2
4	电机驱动	TB6612FNG 模块	1
5	车轮	直径 60mm 橡胶轮，配 3mm 轴	2
6	万向轮（选配）	只在调试初期防侧翻用，正式运行拆除	1
7	锂电池	2S 18650 7.4V 2200mAh 带 2S 保护板	1
8	保险丝	5A 自恢复 PTC（功率路径）	1
9	电阻	4.7kΩ×2（I2C 上拉）、10kΩ×2（电池分压）	若干
10	车架	两层亚克力板 + 铜柱，重心离轴约 0.1m	1
11	USB 转 TTL	CP2102/CH340（921600bps 实测稳定）	1

第 4 章　被控对象建模与控制器设计

本章是全项目的理论核心：先用牛顿-欧拉法把车变成一个四维状态空间里的不稳定矩阵（4.1~4.2），再离线解 LQR 与确定卡尔曼参数（4.3~4.4），设计速度闭环（4.5），最后用仿真验证并对照串级 PID（4.6~4.7）。所有推导保留中间步骤。

4.1 轮式倒立摆动力学建模（完整推导）

表 4-1 模型符号与实测值

符号	含义	实测/取值
M_p, L	车体质量、质心到轮轴高度	1.5 kg, 0.10 m（吊线法）
J_p	车体绕自身质心转动惯量	0.005 kg·m²（摆线法反推）
m, r, J_w	单轮质量、半径、绕轴转动惯量	0.05 kg, 0.03 m, ½mr²
T	双轮合计电机力矩（控制量 u）	—
x, θ	轮轴水平位移（前为正）、车体前倾角	状态量

小角度线性化：$\sin\theta\approx\theta$，$\cos\theta\approx1$；质心位置 $x_c = x + L\cdot\sin\theta \approx x + L\theta$。推导分三步。

第一步：车体水平方向。车体受轮轴水平力 $2f_w$（f_w 为单轮轴力，向前为正），质心水平加速度 $\ddot{x}_c = \ddot{x} + L\cdot\ddot{\theta}$：

$$M_p\cdot(\ddot{x} + L\cdot\ddot{\theta}) = 2\cdot f_w$$

第二步：轮子。每个轮子受地面摩擦 F_f、车体反力 $-f_w$、驱动力矩 T，纯滚动 $\ddot{x} = r\cdot\dot{\omega}_w$。绕轴转动与平动联立：

$$J_w\cdot\ddot{x}/r = T - F_f\cdot r, \qquad m\cdot\ddot{x} = F_f - f_w$$

$$\Longrightarrow\ f_w = T/r - (J_w + m\cdot r^2)\cdot\ddot{x}/r^2$$

代入车体方程并把两轮折算进等效质量，得到第一个独立方程：

$$M_eq\cdot\ddot{x} = 2T/r - M_p\cdot L\cdot\ddot{\theta}, \qquad \text{其中 } M_eq = M_p + 2(J_w + m\cdot r^2)/r^2 \quad (A)$$

第三步：车体绕轮轴的力矩方程。轮轴是非惯性系（有加速度 $\ddot{x}$），对车体加惯性力 $-M_p\cdot\ddot{x}$（作用于质心）。前倾 $\theta>0$ 时重力矩使车继续倒（正方向），惯性力矩使车后仰（负方向）：

$$I_b\cdot\ddot{\theta} = M_p\cdot g\cdot L\cdot\sin\theta - M_p\cdot L\cdot\cos\theta\cdot\ddot{x}$$

线性化：$I_b\cdot\ddot{\theta} = M_p\cdot g\cdot L\cdot\theta - M_p\cdot L\cdot\ddot{x}$，　　其中 $I_b = J_p + M_p\cdot L^2$　(B)

把 (A) 的 $\ddot{x}$ 代入 (B)，整理出两个解耦的二阶方程（记判别式 $D = I_b\cdot M_{eq} - M_p^2L^2$，本参数下 $D=0.0096>0$）：

$$\ddot{\theta} = a_1\cdot\theta + b_1\cdot T,\qquad a_1 = M_{eq}\cdot M_p\cdot g\cdot L/D = 245.8,\qquad b_1 = -2\cdot M_p\cdot L/(r\cdot D) = -1041.7$$

$$\ddot{x} = a_2\cdot\theta + b_2\cdot T,\qquad a_2 = -M_p^2\cdot L^2\cdot g/D = -23.0,\qquad b_2 = 2\cdot I_b/(r\cdot D) = 138.9$$

> **物理自检**　四个系数的正负号比数值更重要：$a_1>0$ 是角度正反馈——“越歪越快歪”的数学形式；$b_1<0$ 表示 $T>0$（向前开）产生向后的惯性力矩扶正车——车前倾必须向前开去“接”它；$a_2<0$ 表示不施力矩时车前倾、轮轴被蹬得后退（动量守恒）；$b_2>0$ 表示正力矩车向前加速。四个号全对，模型才可信。

4.2 状态空间实现与数值验证

取状态 $x = [\theta, \dot{\theta}, s, v]^T$（s 位移，v 车速），得线性状态空间：

开环极点：特征方程 $(s^2-a_1)(s^2-a_2)=0$，得 $s=\pm15.7$ 与 $s=\pm j4.8$。前者即 2.1 节的不稳定实极点（时间尺度 191ms）；后者是位置子系统的无阻尼振荡（车倒下过程中轴前后荡）。能控性：$\mathrm{rank}[B\ AB\ A^2B\ A^3B] = 4$——各列为 $(0,b_1,0,b_2)$、$(b_1,0,b_2,0)$、$(0,a_1b_1,0,a_2b_1)$、$(a_1b_1,0,a_2b_1,0)$，行列式正比于 $b_1b_2(a_1-a_2)\cdot\ldots\neq0$。系统完全可控，LQR 有解。

> **关键联系**　θ、$\dot{\theta}$ 来自卡尔曼滤波，v、s 来自编码器——四个状态全部可测，不需要设计状态观测器。卡尔曼在这里扮演“传感器层面的观测器”，这正是 2.4.3 节 LQR 鲁棒性保证能够兑现的前提。

4.3 LQR 求解与离散化

4.3.1 离线求解（清单 4-1）

在 STM32 上永远不要在线解 Riccati：MATLAB/Octave 离线算 K，板上只做 4 次乘加。权重起点 Q=diag(50,2,1,100)、R=1（整定方向见表 4-2），解得 K = [−2.798, −0.326, −0.094, −0.971]（θ 单位 rad），闭环极点全部拉回稳定域（|eig| ≤ 0.9995，位移模态对应的那对最慢，符合“位置松、姿态紧”的设计意图）。

清单 4-1 code/design/design_lqr.m（Octave/MATLAB，输出即固件参数）

```
%% design_lqr.m — 平衡车 LQR 设计脚本 (Octave/MATLAB 通用)
% 输入: 第3.4节实测参数; 输出: 写入 balance_config.h 的 K
Mp=1.5; L=0.10; Jp=0.005;          % 车体质量/质心高/绕质心惯量(摆线法实测)
m=0.05; r=0.03; Jw=0.5*m*r^2;      % 单轮质量/半径/转动惯量
g=9.8;
Ib=Jp+Mp*L^2;                      % 车体绕轮轴惯量
Meq=Mp+2*(Jw+m*r^2)/r^2;           % 等效质量(车轮折算)
D=Ib*Meq-Mp^2*L^2;                 % 判别式, 必须大于0
a1=Meq*Mp*g*L/D;   b1=-2*Mp*L/(r*D);
a2=-Mp^2*L^2*g/D;  b2=2*Ib/(r*D);
A=[0 1 0 0; a1 0 0 0; 0 0 0 1; a2 0 0 0];
B=[0; b1; 0; b2];
rank(ctrb(A,B))                    % =4 才可控, 否则 LQR 无解
Q=diag([50 2 1 100]);  R=1;        % 起点权重, 整定方向见表4-2
K=lqr(A,B,Q,R)                     % -> [-2.798 -0.326 -0.094 -0.971]
plot(eig(A-B*K),'x')               % 闭环极点须全部在左半平面
```

答辩高频点 K 的元素出现负数不是算错：θ>0（前倾）需要 u>0（向前），而公式是 u=−K·x，所以 K_1 必须为负。自查时在这里误判“算错了”再改成正数，车当场飞出去——这是平衡车调参第一大坑。

4.3.2 离散化：5ms 下连续 K 为什么可以直接用

精确离散（零阶保持）：A_d = e^(AT)，B_d = A^{-1}(A_d − I)B，一行命令 [Ad,Bd]=c2d(A,B,0.005)。若用前向欧拉 A_d≈I+AT，其特征值映射误差量级为 $(sT)^2/2$：对主极点 s=15.7、T=0.005，约 0.31%——可以接受，但这是“连续 K 直接当离散 K 用”的合法性论证，不是“欧拉很精确”的论证。控制周期一旦拉长（如速度环单独降到 20ms），必须改用精确离散并重解离散 Riccati：

$$P_k = Q + A^{T}P_\{k+1\}A - A^{T}P_\{k+1\}B(R + B^{T}P_\{k+1\}B)^{-1}B^{T}P_\{k+1\}A$$ （反向迭代至收敛）

$$K_d = (R + B^{T}PB)^{-1}B^{T}PA$$

本项目 T=5ms，离散 K 与连续 K 差别小于 1%，固件直接使用连续 K，仿真阶段按精确离散校核过闭环极点（图 4-1 的仿真即用 A_d、B_d 做的）。

4.4 卡尔曼滤波器参数确定

三个噪声参数按物理意义确定，不拍脑袋：

- R（观测方差）：车静止放桌上，采 5000 点加速度计 atan2 角算方差。样车实测 $\sigma^2 \approx 0.03°^2$，即 R=0.03；

- Q_angle：陀螺积分模型每步失配方差，与陀螺噪声和 dt 相关，量级 1e-3（度²）；

- Q_bias：零偏随机游走强度，量级 3e-3（度²）。初值可取社区长期验证值，上车后按 7.3 节曲线形态微调。

表 4-2 滤波与控制权重的整定方向表（LQR 的“旋钮”）

动作	效果	过头的症状
KF 的 R↑（更信陀螺）	输出平滑、抗加速度计振动	相位滞后增大 → 车低频大幅慢摆
KF 的 R↓（更信加速度计）	零偏跟踪快、滞后小	振动噪声直接进 θ，电机嗡嗡抖
Q_1（θ 权重）↑	回正快、抗扰强	高频抖动，驱动发热
Q_2（θ̇ 权重）↑	阻尼感强、动作稳	反应迟钝
Q_4（v 权重）↑	抗推、速度定位准	起停激烈、速度超调
R_ctrl（控制权重）↑	整体温柔省电	软、一推就踉跄

整定顺序：先用实测参数离线解 K 并仿真确认（4.6），上车后只动两个旋钮——Q_1 与 R_ctrl，方向按上表；最后调速度环，调完回头微调 Q_2 补阻尼。滤波权重的调整与控制权重独立进行，但每改一次滤波都要重验整车——滤波滞后吃掉的是控制环的相位裕度。

4.5 速度闭环结构设计

4.5.1 串级结构：速度 PI 输出目标角偏置

由 2.5 节，加减速必须先倾斜。因此速度误差不直接给力矩，而是给直立环一个目标角偏置：设速度误差 $e = v - v_ref$（超速为正），需要目标角 $\theta_t = -k \cdot e$（超

速则负目标即后仰）。直立环按误差 $(\theta_meas - \theta_t) = (\theta_meas + k\cdot e)$ 工作，故实现上把 PI 输出加到测量角上：

$$bias = Kp_v\cdot e + Ki_v\cdot\int e\cdot dt \quad （限幅 \ |bias| \le 8°，带抗积分饱和）$$

$$u = -K\cdot[\theta_meas + bias,\ \theta,\ s,\ v]^T$$

稳态行为自检：车速持续超调 → bias>0 → 直立环把车压到 θ≈−bias（后仰）→ 减速 → e 回零 → bias 回零 → 车回正。“想停先仰、想走先倾”由结构保证，而不是靠 PID 硬顶。积分项负责吃掉两类恒定扰动：坡度的重力分量与电机死区。

> **大白话类比** 速度环像坐在车顶的指挥官：它不直接碰方向盘（力矩），只对直立环喊“目标姿态是后仰 3°”。喊多少？超速越多喊得越狠（P），喊了还没效果就逐拍加码（I）。

4.5.2 备选：增广积分 LQR（LQI）

把速度误差积分升为第五个状态 $x_5 = \int(v - v_ref)dt$，即 $\dot{x}_5 = v - v_ref$，增广矩阵后重解一次 lqr()：

LQI 让“消静差”进入最优框架，积分增益由 Q_5 统一权衡（Q_5 大消差快但易超调）。工程路线建议：先用串级 PI 把车立稳（结构简单、各环独立可调），有精力再上 LQI 对比，这本身就是论文里一节现成的实验内容。

4.5.3 编码器速度量化：一个必须算的数字

13 线 × 4 倍频 × 减速比 30 = 输出轴每转 1560 计数。5ms 窗口内 1 个计数对应 $\Delta v = 2\pi r/(1560\times0.005) \approx 0.024$ m/s。目标速度 0.3m/s 时量化噪声高达 8%——直接进微分性质的通道必然抖。对策按优先级：① 速度低通一阶滤波（系数 0.25，即约 4 拍时间常数，滞后 20ms 量级，仍在裕量内）；② 需要更平滑时把速度窗拉长到 10ms；③ 花钱买安静：换高线数编码器。

4.6 仿真验证

上车前先在仿真里回答两个问题：闭环是否真的稳定？先仰后动的轨迹是否符合 4.5 的预期？清单 4-2 用精确离散 + 迭代离散 Riccati 做闭环仿真，图 4-1 即其输出：初始 3° 扰动在约 0.6s 内回零，速度先正向冲出（向前接车）再回零，力矩未触及 ±0.6N·m 限幅。

清单 4-2　code/design/sim_lqr.py（闭环恢复仿真，图 4-1 由本脚本生成）

```
# sim_lqr.py — LQR 闭环恢复仿真（numpy 即可，无需控制工具箱）
import numpy as np
Mp,L,Jp,m,r,g = 1.5,0.1,0.005,0.05,0.03,9.8
Ib=Jp+Mp*L**2; Meq=Mp+2*(0.5*m*r**2+m*r**2)/r**2; D=Ib*Meq-Mp**2*L**2
a1=Meq*Mp*g*L/D; b1=-2*Mp*L/(r*D); a2=-Mp**2*L**2*g/D; b2=2*Ib/(r*D)
A=np.array([[0,1,0,0],[a1,0,0,0],[0,0,0,1],[a2,0,0,0]])
B=np.array([[0],[b1],[0],[b2]])
# 精确离散：矩阵指数级数展开
def expm(M):
    X=np.eye(M.shape[0]); t=np.eye(M.shape[0])
    for k in range(1,40): t=t@M/k; X=X+t
    return X
T=0.005; Z=np.zeros((5,5)); Z[:4,:4]=A*T; Z[:4,4:]=B*T
E=expm(Z); Ad,Bd=E[:4,:4],E[:4,4]
# 迭代离散 Riccati
Q=np.diag([50.,2,1,100]); R=np.array([[1.]])
P=Q.copy()
for _ in range(50000):
    K=np.linalg.solve(R+B.T@P@B, B.T@P@A)
    Pn=Q+A.T@P@(A-B@K)
    if np.max(np.abs(Pn-P))<1e-10: P=Pn; break
    P=Pn
K=np.linalg.solve(R+B.T@P@B, B.T@P@A)
# 闭环仿真：初始 3 度，力矩限幅 0.6 N.m
x=np.array([3*np.pi/180,0,0,0]); log=[]
for k in range(600):
    u=np.clip(-(K@x)[0], -0.6, 0.6)
    log.append([x[0]*180/np.pi, x[3], u]); x=Ad@x+Bd*u
```

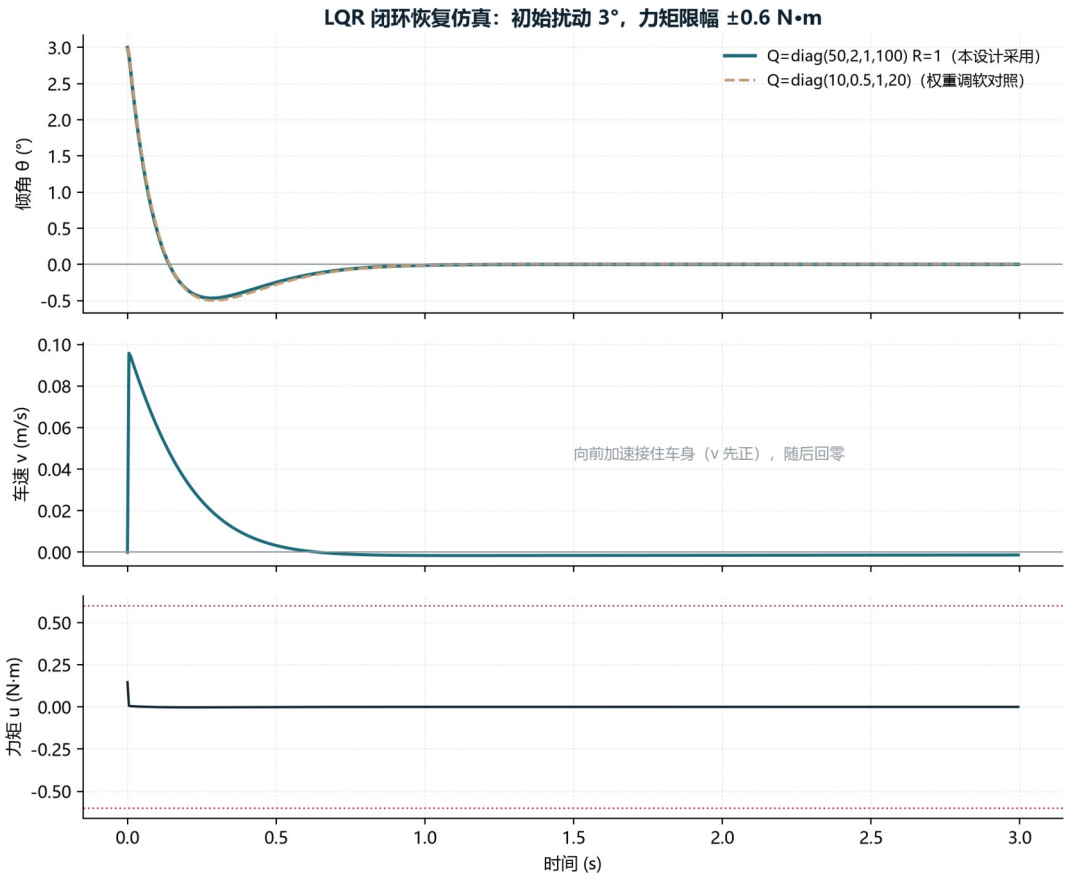


图 4-1 LQR 闭环恢复仿真：初始扰动 3°，两种 Q 权重对比（本设计采用 Q=diag(50,2,1,100)）

4.7 与串级 PID 的系统对比

经典三环 PID 架构是：直立环 PD（$u = Kp \cdot e_\theta + Kd \cdot \dot{\theta}$）+ 速度环 PI + 转向环 PD。表 4-3 给出系统对照，每一行都是可答辩的技术点。

表 4-3 三环 PID 与 LQR 状态反馈对照

维度	三环 PID	LQR 状态反馈
结构	三个独立回路人工串联	单回路全状态反馈
参数来源	N 个增益逐环试凑	Q/R 权重（维度低、有物理含义），K 一次解出
耦合处理	环间耦合靠试凑兜住	状态耦合显式进 A 矩阵，理论自动权衡
静差	I 项消除	纯 LQR 无积分有静差；增广 LQI 或串级 PI 补
鲁棒性	无一般保证	全状态反馈下 ≥60° 相位裕度、≥6dB 增益裕度

维度	三环 PID	LQR 状态反馈
换车移植	全部重调	改模型参数重解 K，方法论不变
计算量	极小	同样极小（板上 4 次乘加）
模型需求	不需要模型	需要可实测的模型（第 3.4 节半天工作量）

本质联系：直立环 PD 的 $Kp \cdot e_\theta + Kd \cdot \dot{\theta}$ 恰好等于 LQR 增益的前两项 $-K_1\theta - K_2\dot{\theta}$；速度环 PI 约等于 LQR 的 v 反馈项加积分增广。PID 是 LQR 最优解的人工低配版——你在 PID 上调几天试出来的那组 Kp/Kd 比例，LQR 从 Q/R 一次算出，还附赠裕度保证。PID 的真实优势只剩一条：不需要模型。有模型、且模型可实测，就应该上 LQR。

第 5 章　STM32 固件设计（含完整代码）

5.1 工程结构、开发环境与外设分配

开发环境：STM32CubeMX（引脚与时钟初始化）+ Keil MDK AC6 或 CMake + arm-none-eabi-gcc，编译开 -O2 与硬件 FPU（-mfpu=fpv4-sp-d16 -mfloat-abi=hard），全工程用 float 不用 double——M4F 的单精度 FPU 上 double 要走软件库，慢一个数量级以上。时钟树：HSE 8MHz 倍频到 SYSCLK=168MHz，APB1=42MHz（定时器×2=84MHz），APB2=84MHz。模块划分见表 5-1，每个模块单一职责、可独立测试。

表 5-1 固件模块划分

模块	文件	职责
参数配置	balance_config.h	采样率/增益/限幅等全局唯一参数源，注释给出每项的理论出处
姿态驱动	mpu6050.c/.h	I2C 初始化、14 字节突发读、上电零偏标定、总线恢复
姿态解算	attitude.c/.h	二维卡尔曼滤波（2.3 节五公式的 C 映射）
测速	encoder.c/.h	TIM3/TIM4 编码器模式、差分测速、一阶低通、位移累计
控制	controller.c/.h	LQR 状态反馈 + 速度 PI（4.3/4.5 节的实现）
执行	motor.c/.h	TB6612 方向/PWM、死区补偿、电池电压归一化
遥测	uart_frame.c/.h	AA 55 帧 + CRC16-MODBUS，DMA 发送
调度	main.c	时钟/外设初始化、启动 gating、5ms 控制中断、看门狗

5.2 控制周期与时序预算

全车脊柱是 TIM6 的 5ms 更新中断：读 MPU6050 → 卡尔曼 → 编码器 → LQR+速度环 → PWM，一次不落全在中断内完成，见图 5-1。最重的开销是 I2C 同步读 14 字节约 0.4ms（400kHz 含地址帧与寄存器指针），占周期 8%；其余计算合计不足 0.05ms。两条纪律：① 中断内任何调用不得阻塞无上限（所有 HAL I2C 调

用带 4ms 超时，失败立即断力矩）；② 遥测组帧与发送放主循环或 DMA，不占控制时间。

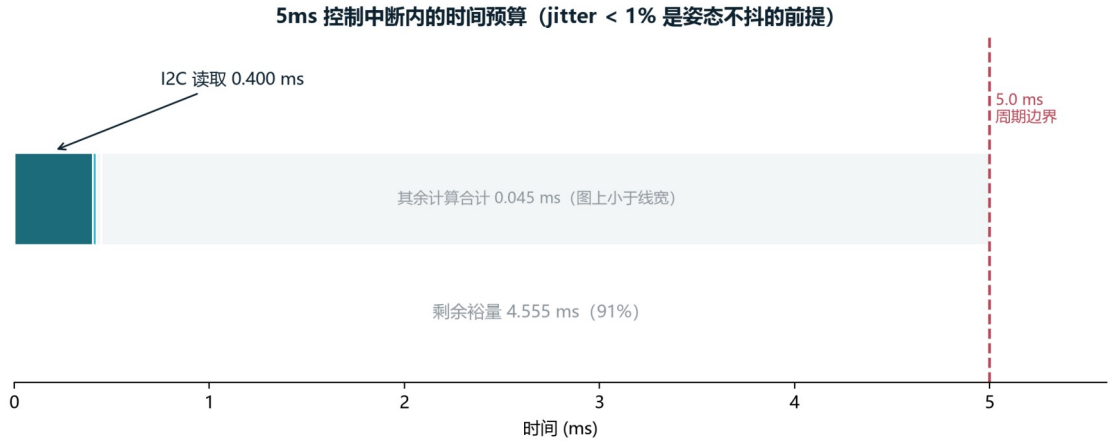


图 5-1 5ms 控制中断内的时间预算（GPIO 翻转 + 示波器实测口径）

5.3 全局参数配置

清单 5-1 是全工程唯一参数源。注意两个“看起来像错的”参数：K_THETA 为负（4.3 节负号解释）与 DEADZONE 为正数（实测死区，7.3 节标定）。每个数值都标注了理论出处章节。

清单 5-1 code/stm32/balance_config.h（全局参数）

```
/**
 * balance_config.h — 系统核心参数（全工程唯一参数源）
 * 平台: STM32F407VET6 + MPU6050 + TB6612FNG + GA25-370(1:30, AB 相 13 线)
 * 每个数值的推导/标定出处见注释；改参数只改这里
 */
#ifndef BALANCE_CONFIG_H
#define BALANCE_CONFIG_H

/* ---- 控制周期 ---- */
#define CTRL_DT        0.005f     /* 5ms = 200Hz, TIM6 更新中断; 依据见 2.1 节 */

/* ---- LQR 增益: 清单 4-1 离线解出, Q=diag(50,2,1,100) R=1, θ 单位 rad ---- */
#define K_THETA  (-2.798f)        /* 负号不是错误! θ>0 前倾需 u>0 向前, u=-K·x */
#define K_THDOT  (-0.326f)        /* 单位: N·m / (rad/s) */
#define K_POS    (-0.094f)        /* 单位: N·m / m */
#define K_VEL    (-0.971f)        /* 单位: N·m / (m/s) */

/* ---- 卡尔曼滤波(单位: 度), 标定方法见 4.4 节 ---- */
#define KF_Q_ANGLE  0.001f        /* 过程噪声: 陀螺积分失配方差 */
#define KF_Q_BIAS   0.003f        /* 零偏随机游走强度 */
#define KF_R_MEAS   0.03f         /* 观测噪声: acc 角方差(静止实测) */

/* ---- 速度环 PI(输出: 目标角偏置, 度), 结构见 4.5 节 ---- */
```

```
#define VEL_KP       0.9f
#define VEL_KI       0.35f
#define VEL_OUT_MAX  8.0f          /* 偏置限幅：直立环能力的约 1/3 */
#define VEL_LP_A     0.25f         /* 编码器速度一阶低通(4.5.3 节量化分析) */

/* ---- 执行器 ---- */
#define PWM_PERIOD   4199U         /* 84MHz/20kHz-1，TIM1 ARR (3.2.2 节) */
#define U_MAX        0.95f         /* 归一化输出限幅 */
#define DEADZONE     0.06f         /* 实测死区占空比 6%，7.3 节标定 */
#define TORQUE_FULL  0.55f         /* 归一化 1.0 对应的双轮合计力矩 N·m */
#define VBAT_NOM     7.4f          /* 电池标称电压，输出归一化基准 */

/* ---- 传感 ---- */
#define WHEEL_R      0.03f         /* 轮半径 m */
#define ENC_PPR      1560.0f       /* 13 线×4 倍频×30 减速比 */
#define GYRO_LSB     16.4f         /* ±2000dps: 16.4 LSB/(°/s) */
#define ACC_LSB      16384.0f      /* ±2g: 16384 LSB/g */

/* ---- MPU6050 寄存器 ---- */
#define MPU_ADDR       (0x68U << 1)
#define MPU_WHO_AM_I   0x75U
#define MPU_PWR_MGMT1  0x6BU
#define MPU_DLPF_CFG   0x1AU
#define MPU_GYRO_CFG   0x1BU
#define MPU_ACCEL_CFG  0x1CU
#define MPU_DATA       0x3BU

#endif /* BALANCE_CONFIG_H */
```

5.4 MPU6050 驱动

三个必问细节都在清单 5-2 里：WHO_AM_I 自检失败时先做 9 个 SCL 脉冲的总线恢复再报错（7.4 故障 6）；上电 400 样本静止标零偏，标定期间车必须躺着不动——运动中标会把动态角速度吸进零偏，之后所有姿态全错；14 字节一次事务突发读保证同帧一致（2.2.1 节）。

清单 5-2 code/stm32/mpu6050.c（底层驱动）

```
/**
 * mpu6050.c — MPU6050 驱动：初始化 / 突发读取 / 零偏标定 / 总线恢复
 * I2C1: PB8=SCL, PB9=SDA, 400kHz, 上拉 4.7k(计算见 3.2.3)
 */
#include "mpu6050.h"
#include "balance_config.h"

extern I2C_HandleTypeDef hi2c1;
extern void Debug_Fault(uint8_t code);       /* 故障码 -> 遥测/LED */

static float s_gyro_bias = 0.0f;             /* 上电标定的陀螺零偏(°/s) */

/* I2C 死锁恢复：9 个 SCL 脉冲释放从机 SDA，再重新初始化外设 (7.4 故障 6) */
```

```
static void i2c_bus_recover(void)
{
    GPIO_InitTypeDef io = {0};
    io.Pin = GPIO_PIN_8; io.Mode = GPIO_MODE_OUTPUT_OD; io.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOB, &io);
    for (int i = 0; i < 9; i++) {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_RESET);
        for (volatile int d = 0; d < 500; d++) { }
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_SET);
        for (volatile int d = 0; d < 500; d++) { }
    }
    HAL_I2C_DeInit(&hi2c1);
    HAL_I2C_Init(&hi2c1);
}

static HAL_StatusTypeDef mpu_write(uint8_t reg, uint8_t val)
{
    return HAL_I2C_Mem_Write(&hi2c1, MPU_ADDR, reg, 1, &val, 1, 4);
}

uint8_t MPU_Init(void)
{
    uint8_t id = 0;
    if (HAL_I2C_Mem_Read(&hi2c1, MPU_ADDR, MPU_WHO_AM_I, 1, &id, 1, 10) != HAL_OK
        || id != 0x68) {
        i2c_bus_recover();
        return 0;                           /* 自检失败：上层停机 */
    }
    mpu_write(MPU_PWR_MGMT1, 0x01);         /* 唤醒，时钟=PLL(陀螺) */
    HAL_Delay(10);
    mpu_write(MPU_DLPF_CFG, 0x03);          /* 内部低通 42Hz (2.2.1 节) */
    mpu_write(MPU_GYRO_CFG, 0x18);          /* ±2000dps */
    mpu_write(MPU_ACCEL_CFG, 0x00);         /* ±2g */

    /* 零偏标定：上电必须静止，400 样本均值；运动中标定=污染零偏 */
    uint8_t b[14];
    float sum = 0.0f;
    for (int i = 0; i < 400; i++) {
        if (HAL_I2C_Mem_Read(&hi2c1, MPU_ADDR, MPU_DATA, 1, b, 14, 4) != HAL_OK)
            return 0;
        int16_t gy = (int16_t)((b[10] << 8) | b[11]);
        sum += (float)gy / GYRO_LSB;
        HAL_Delay(2);
    }
    s_gyro_bias = sum / 400.0f;
    return 1;
}

/* 突发读 14 字节：一次事务保证加速度/陀螺属于同一采样时刻 */
uint8_t MPU_Read(MpuFrame *f)
{
    uint8_t b[14];
    if (HAL_I2C_Mem_Read(&hi2c1, MPU_ADDR, MPU_DATA, 1, b, 14, 4) != HAL_OK)
        return 0;                           /* 调用方立即断力矩 */
    f->ax_g  = (float)(int16_t)((b[0] << 8) | b[1]) / ACC_LSB;
    f->az_g  = (float)(int16_t)((b[4] << 8) | b[5]) / ACC_LSB;
    f->gyro_dps = (float)(int16_t)((b[10] << 8) | b[11]) / GYRO_LSB - s_gyro_bias;
    return 1;
```

```
}

float MPU_GetBias(void) { return s_gyro_bias; }
```

5.5 卡尔曼姿态解算

清单 5-3 是 2.3.3 节五条公式的逐行 C 映射，没有一行没有出处。两个工程细节 θ̇ 直接用去零偏陀螺值而不是卡尔曼微分（微分放大噪声）；观测角 atan2(-ax, az) 的负号对应“前倾为正”约定，换安装方向就换号（7.3 节符号自检）。

清单 5-3 code/stm32/attitude.c（二维卡尔曼）

```
/**
 * attitude.c — 二维卡尔曼姿态解算: 状态 [θ(°), 陀螺零偏(°/s)]
 * 推导见 2.3 节: 预测=陀螺积分, 观测=加速度计 atan2 角, K 按方差比分配
 */
#include "attitude.h"
#include "balance_config.h"
#include <math.h>

static float s_theta = 0.0f, s_bias = 0.0f;
static float s_P[2][2] = {{1.0f, 0.0f}, {0.0f, 1.0f}};
static float s_gyro_raw = 0.0f;

void Att_Update(float gyro_dps, float ax_g, float az_g)
{
    s_gyro_raw = gyro_dps;
    float z = atan2f(-ax_g, az_g) * 57.29578f;   /* 观测: 前倾为正(°) */

    /* ---- 预测(公式①②) ---- */
    s_theta += CTRL_DT * (gyro_dps - s_bias);
    float p11 = s_P[0][0] + CTRL_DT * (CTRL_DT * s_P[1][1]
              - s_P[0][1] - s_P[1][0]) + KF_Q_ANGLE;
    float p12 = s_P[0][1] - CTRL_DT * s_P[1][1];
    float p21 = s_P[1][0] - CTRL_DT * s_P[1][1];
    float p22 = s_P[1][1] + KF_Q_BIAS;

    /* ---- 更新(公式③④⑤) ---- */
    float S  = p11 + KF_R_MEAS;                  /* 新息方差 */
    float K1 = p11 / S, K2 = p21 / S;            /* 增益 */
    float y  = z - s_theta;                      /* 新息 */
    s_theta += K1 * y;
    s_bias  += K2 * y;                           /* 零偏在线自学习 */

    s_P[0][0] = p11 - K1 * p11;  s_P[0][1] = p12 - K1 * p12;
    s_P[1][0] = p21 - K2 * p11;  s_P[1][1] = p22 - K2 * p12;
}

float Att_GetTheta(void)   { return s_theta; }
float Att_GetThetaDot(void){ return s_gyro_raw; }  /* θ直接用陀螺, 免微分噪声 */
float Att_GetBias(void)    { return s_bias; }
```

5.6 编码器测速

清单 5-4 code/stm32/encoder.c（测速与位移）

```
/**
 * encoder.c — TIM3/TIM4 编码器模式(4倍频) -> 车速/位移
 * 量化分析见 4.5.3: 5ms 内 1 计数 = 0.024m/s, 必须低通
 */
#include "encoder.h"
#include "balance_config.h"

extern TIM_HandleTypeDef htim3, htim4;

static float v_lp = 0.0f, s_pos = 0.0f;

void Encoder_Start(void)
{
    HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
    HAL_TIM_Encoder_Start(&htim4, TIM_CHANNEL_ALL);
}

float Encoder_GetSpeed(void)          /* m/s, 左右轮平均, 已低通 */
{
    int32_t dn = (int16_t)__HAL_TIM_GET_COUNTER(&htim3)   /* 强转16位: 自动回绕 */
               + (int16_t)__HAL_TIM_GET_COUNTER(&htim4);
    __HAL_TIM_SET_COUNTER(&htim3, 0);
    __HAL_TIM_SET_COUNTER(&htim4, 0);
    float v = 2.0f * 3.14159f * WHEEL_R * (float)dn
            / (ENC_PPR * 0.005f);
    v_lp  += VEL_LP_A * (v - v_lp);   /* 一阶低通: 压量化噪声(4.5.3) */
    s_pos += v_lp * CTRL_DT;
    return v_lp;
}

float Encoder_GetPos(void) { return s_pos; }
void  Encoder_ResetPos(void) { s_pos = 0.0f; }
```

5.7 控制器：LQR + 速度环

清单 5-5 code/stm32/controller.c

```
/**
 * controller.c — LQR 状态反馈 + 速度 PI 串级(4.3/4.5 节的实现)
 * 结构: bias=PI(v-v_ref); u = -K·[θ+bias, θ̇, s, v]
 *       bias>0 等价于目标后仰 -> 减速; 反之加速。限幅+抗饱和。
 */
#include "controller.h"
#include "balance_config.h"

static float s_i_vel = 0.0f;

static float velocity_loop(float e)             /* e = v - v_ref, 超速为正 */
{
    s_i_vel += VEL_KI * e * CTRL_DT;            /* 积分: 吃坡度/死区静差 */
    if (s_i_vel >  VEL_OUT_MAX) s_i_vel =  VEL_OUT_MAX;   /* 抗饱和 */
```

```
    if (s_i_vel < -VEL_OUT_MAX) s_i_vel = -VEL_OUT_MAX;
    float bias = VEL_KP * e + s_i_vel;
    if (bias >  VEL_OUT_MAX) bias =  VEL_OUT_MAX;
    if (bias < -VEL_OUT_MAX) bias = -VEL_OUT_MAX;
    return bias;                                /* 单位: 度 */
}

void Controller_ResetIVel(void) { s_i_vel = 0.0f; }

float Controller_Update(float theta_deg, float thdot_rps,
                        float s, float v, float v_ref)
{
    float bias_deg = velocity_loop(v - v_ref);
    float th = (theta_deg + bias_deg) * 0.0174533f;   /* 度->rad, 不可省 */
    float u = -(K_THETA * th + K_THDOT * thdot_rps
              + K_POS * s + K_VEL * v);         /* N·m, 双轮合计 */
    return u;
}
```

5.8 电机驱动与死区补偿

清单 5-6 code/stm32/motor.c

```
/**
 * motor.c — TB6612 驱动: 力矩指令 -> PWM
 * 死区补偿(7.4故障3) + 电池电压归一化(7.4故障8)
 * 接线: AIN1/PB12 AIN2/PB13 BIN1/PB14 BIN2/PB15, PWMA=TIM1_CH1 PWMB=TIM1_CH2
 */
#include "motor.h"
#include "balance_config.h"
#include <math.h>

extern TIM_HandleTypeDef htim1;
extern ADC_HandleTypeDef hadc1;

static float vbat_norm(void)              /* VBAT_NOM / VBAT, 分压1/11 */
{
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 1);
    float vbat = (float)HAL_ADC_GetValue(&hadc1) * 3.3f / 4095.0f * 11.0f;
    return (vbat > 5.0f) ? VBAT_NOM / vbat : 1.0f;
}

static float deadband_comp(float d)       /* 小信号区抬到死区之上 */
{
    float mag = fabsf(d);
    float out = DEADZONE + (U_MAX - DEADZONE) * mag / U_MAX;
    return (d < 0.0f) ? -out : out;
}

static void ch_out(GPIO_TypeDef *p1, uint16_t g1,
                   GPIO_TypeDef *p2, uint16_t g2,
                   float d, uint32_t ch)
{
    uint32_t duty = (uint32_t)(fabsf(d) * PWM_PERIOD);
    GPIO_PinState s = (d >= 0.0f) ? GPIO_PIN_SET : GPIO_PIN_RESET;
    HAL_GPIO_WritePin(p1, g1, s);
```

```
    HAL_GPIO_WritePin(p2, g2, (GPIO_PinState)(s ^ GPIO_PIN_1));
    __HAL_TIM_SET_COMPARE(&htim1, ch, duty);
}

void Motor_SetTorque(float u)             /* u: 双轮合计力矩 N·m */
{
    float d = u / TORQUE_FULL;            /* 归一化 */
    d *= vbat_norm();                     /* 电压前馈: 出力不随电量缩水 */
    if (d >  U_MAX) d =  U_MAX;
    if (d < -U_MAX) d = -U_MAX;
    d = deadband_comp(d);
    ch_out(GPIOB, GPIO_PIN_12, GPIOB, GPIO_PIN_13, d, TIM_CHANNEL_1);
    ch_out(GPIOB, GPIO_PIN_14, GPIOB, GPIO_PIN_15, d, TIM_CHANNEL_2);
}

void Motor_Start(void)
{
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
}
```

5.9 遥测帧协议

协议设计原则与工业遥测的通行做法一致：定界靠帧头、完整性靠 CRC、有序性靠 SEQ。遥测帧 100Hz（每 2 个控制拍发一帧），载荷 5 个 float（θ、θ̇、v、u、KF 零偏），27 字节/帧 → 净吞吐约 27kbps，921600bps 占用 3%，余量充足。上位机下发参数帧 TYPE=0x02（v_ref、Kp、Ki），供在线调参。

清单 5-7 code/stm32/uart_frame.c（遥测组帧 + CRC16）

```
/**
 * uart_frame.c — 遥测帧: AA 55 | TYPE | LEN | SEQ | PAYLOAD | CRC16(低前)
 * USART1 921600 8N1, DMA 发送; CRC 对 TYPE..PAYLOAD 全部字节
 */
#include "uart_frame.h"
#include <string.h>

extern UART_HandleTypeDef huart1;
static uint8_t s_seq = 0, s_div = 0;

static uint16_t crc16_modbus(const uint8_t *p, uint32_t n)
{
    uint16_t c = 0xFFFF;
    while (n--) {
        c ^= *p++;
        for (int i = 0; i < 8; i++)
            c = (c & 1U) ? (uint16_t)((c >> 1) ^ 0xA001U) : (uint16_t)(c >> 1);
    }
    return c;
}

/* 载荷: θ(°) θ̇(rad/s) v(m/s) u(N·m) KF零偏(°/s) — 5×float=20 字节 */
void Telem_Send(float th, float thd, float v, float u, float bias)
{
```

```
    if (++s_div < 2) return;              /* 200Hz -> 100Hz */
    s_div = 0;
    static uint8_t fr[27];
    fr[0] = 0xAA; fr[1] = 0x55; fr[2] = 0x01; fr[3] = 20; fr[4] = s_seq++;
    memcpy(&fr[5],  &th,   4);  memcpy(&fr[9],  &thd,  4);
    memcpy(&fr[13], &v,    4);  memcpy(&fr[17], &u,    4);
    memcpy(&fr[21], &bias, 4);
    uint16_t crc = crc16_modbus(&fr[2], 23);
    fr[25] = (uint8_t)(crc & 0xFF); fr[26] = (uint8_t)(crc >> 8);
    HAL_UART_Transmit_DMA(&huart1, fr, sizeof fr);
}
```

5.10 主程序：启动 gating 与故障安全

上电三步走：外设初始化 → MPU 自检+零偏标定 → 等 gating 条件。gating 要求陀螺安静（|gyro|<5°/s）且车体接近竖直（|θ|<2°）持续 200ms 才进入闭环——上电时车躺着就直接开环是新手第一炸。故障安全两条：I2C 读失败立即零力矩；独立看门狗 500ms，主循环卡死即停机。

清单 5-8 code/stm32/main.c（调度核心）

```
/**
 * main.c — TIM6 5ms 控制中断(硬实时) + 主循环(gating/看门狗/在线参数)
 * 外设: I2C1(MPU) TIM1(20kHz PWM) TIM3/4(编码器) TIM6(5ms) USART1(遥测)
 */
#include "stm32f4xx_hal.h"
#include "balance_config.h"
#include "mpu6050.h"
#include "attitude.h"
#include "encoder.h"
#include "controller.h"
#include "motor.h"
#include "uart_frame.h"
#include <math.h>

I2C_HandleTypeDef hi2c1;
TIM_HandleTypeDef htim1, htim3, htim4, htim6;
UART_HandleTypeDef huart1;
IWDG_HandleTypeDef hiwdg;

static volatile uint8_t g_run = 0;     /* gating 通过后才闭环 */
static float g_v_ref = 0.0f;           /* 目标速度，上位机参数帧下发 */
static uint16_t g_idle = 0;

/* ---- 5ms 心跳: 控制环在中断内一次做完，不允许回主循环再算 ---- */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *h)
{
    if (h->Instance != TIM6) return;
    MpuFrame f;
    if (!g_run || !MPU_Read(&f)) {     /* 未就绪/I2C 失败: 故障安全 */
        Motor_SetTorque(0.0f);
        return;
    }
    Att_Update(f.gyro_dps, f.ax_g, f.az_g);
```

```
    float v = Encoder_GetSpeed();
    float u = Controller_Update(Att_GetTheta(), Att_GetThetaDot() * 0.0174533f,
                                Encoder_GetPos(), v, g_v_ref);
    Motor_SetTorque(u);
    Telem_Send(Att_GetTheta(), Att_GetThetaDot() * 0.0174533f, v, u,
               Att_GetBias());
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();              /* CubeMX 生成: 168MHz */
    MX_GPIO_Init(); MX_I2C1_Init(); MX_TIM1_Init();
    MX_TIM3_Init(); MX_TIM4_Init(); MX_TIM6_Init();
    MX_USART1_UART_Init(); MX_ADC1_Init();

    if (!MPU_Init()) Error_Loop();     /* WHO_AM_I/零偏标定失败即停 */
    Encoder_Start();
    Motor_Start();

    hiwdg.Instance = IWDG;             /* 独立看门狗 ~500ms */
    hiwdg.Init.Prescaler = IWDG_PRESCALER_32;
    hiwdg.Init.Reload    = 625;
    HAL_IWDG_Init(&hiwdg);

    HAL_TIM_Base_Start_IT(&htim6);     /* 启动 200Hz 心跳 */
    while (1) {
        /* 启动 gating: 陀螺安静 + 车体近竖直, 持续 200ms 才放手 */
        if (!g_run && fabsf(Att_GetThetaDot()) < 5.0f
                   && fabsf(Att_GetTheta())  < 2.0f) {
            if (++g_idle > 40) {       /* 40×5ms = 200ms */
                g_run = 1;
                Controller_ResetIVel();
            }
        } else if (!g_run) {
            g_idle = 0;
        }
        Param_Poll(&g_v_ref);          /* 上位机参数帧解析(6.3 节) */
        HAL_IWDG_Refresh(&hiwdg);
    }
}

void Error_Loop(void) { Motor_SetTorque(0.0f); while (1) { } }
```

5.11 编译、烧录与常见坑

- 必须开 -O2 与 FPU 硬浮点选项；用 printf 调试时 float 格式在 Keil MicroLIB 下需勾选“Use MicroLIB”或自实现 _printf_float；

- 烧录用 SWD（ST-Link），不要用串口 ISP——调试期要反复刷机并打断点；

- 第一次闭环前，先用 5.9 节遥测帧在串口助手里确认四个量的符号与量级（2.2 节符号自检的固件版）；

- 中断优先级：TIM6 设最高（控制节拍不可被打断），USART1 DMA 次之，其余最低；HAL_Delay 在中断里是死等 SysTick，控制中断里严禁调用；

- mpu6050.h/attitude.h 等接口头文件随清单配套，函数签名与本文一致即可对上；

第 6 章　上位机与调参工具（含完整代码）

6.1 开发环境与依赖

Python 3.10+，依赖四个包：pyserial（串口）、numpy（数据缓冲）、PyQt5（界面框架）、pyqtgraph（实时曲线）。一键安装：pip install pyserial numpy PyQt5 pyqtgraph -i https://pypi.tuna.tsinghua.edu.cn/simple。职责划分：串口读取是阻塞 IO，放 QThread；帧到达经信号槽投递 UI 线程（Qt 跨线程信号自动转为队列连接，免手工加锁）；曲线用 pyqtgraph——它基于 Qt 原生对象与 numpy 直通，重绘只更新数据指针，100Hz 刷新无压力，matplotlib 每次走完整渲染管线做这件事会卡。

6.2 串口协议解析（状态机）

串口本质是字节流，粘包、半包、误码、假帧头都会出现。解析器唯一正确的写法：维护字节缓冲，每次尽力提取；找不到完整帧就留着等下次；长度非法或 CRC 错就丢弃并计数，从下一字节重扫。清单 6-1 的解析器对任意切分方式结果一致，这是它通过对抗性测试（垃圾字节+粘包+半包混合灌入）的依据。

清单 6-1　code/host/parser.py（流式帧解析）

```
# parser.py — 遥测帧流式解析：对抗粘包/半包/假帧头，任意切分结果一致
import struct

HEADER = b"\xAA\x55"

def crc16(data: bytes) -> int:
    crc = 0xFFFF
    for byte in data:
        crc ^= byte
        for _ in range(8):
            crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
    return crc

class TelemParser:
    def __init__(self):
        self.buf = bytearray()
        self.bad_frames = 0          # 误码/假帧头计数 -> 线缆质量指标

    def feed(self, data: bytes):
        self.buf += data
        while True:
            i = self.buf.find(HEADER)
            if i < 0:
                del self.buf[:-1]    # 保留最后1字节(可能是帧头一半)
                return
            if i:
```

```
                del self.buf[:i]
            if len(self.buf) < 6:
                return              # 帧头+TYPE+LEN+SEQ+CRC 未齐
            typ, ln = self.buf[2], self.buf[3]
            total = 4 + ln + 2
            if len(self.buf) < total:
                return              # 半包：等下次数据
            frame = bytes(self.buf[:total])
            del self.buf[:total]
            if typ != 0x01 or ln != 20:
                self.bad_frames += 1
                continue            # 假帧头：丢弃重扫
            payload = frame[4:4 + ln]
            crc_recv = frame[4 + ln] | (frame[5 + ln] << 8)
            if crc16(frame[2:4 + ln]) != crc_recv:
                self.bad_frames += 1
                continue            # 误码帧：丢弃并计数
            th, thd, v, u, bias = struct.unpack("<5f", payload)
            yield th, thd, v, u, bias
```

6.3 实时曲线与在线调参面板

清单 6-2 的上位机做两件事：四通道实时曲线（θ、v、u、零偏）与参数下发（v_ref、速度环 Kp/Ki，TYPE=0x02 帧）。第 7 章的每一次整定都在这个界面上完成：改参数→看曲线→归因。

清单 6-2 code/host/gui.py（PyQt5 + pyqtgraph）

```
# gui.py — 实时遥测与在线调参上位机
import sys, struct
import serial
from PyQt5 import QtCore, QtWidgets
import pyqtgraph as pg
from parser import TelemParser, crc16

class SerialThread(QtCore.QThread):
    frame_ready = QtCore.pyqtSignal(object)
    def __init__(self, port):
        super().__init__()
        self.ser = serial.Serial(port, 921600, timeout=0.05)
        self.parser = TelemParser()
        self.running = True
    def run(self):
        while self.running:
            data = self.ser.read(256)
            if data:
                for frame in self.parser.feed(data):
                    self.frame_ready.emit(frame)

class MainWindow(QtWidgets.QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("平衡车遥测调参 V1.0")
        central = QtWidgets.QWidget(); self.setCentralWidget(central)
        layout = QtWidgets.QVBoxLayout(central)
```

```
        self.plot = pg.GraphicsLayoutWidget(); layout.addWidget(self.plot, 6)
        panel = QtWidgets.QHBoxLayout(); layout.addLayout(panel, 1)
        self.data, self.curves, self.idx = {}, {}, 0
        for row, (name, unit, color) in enumerate([
                ("theta", "deg", "b"), ("v", "m/s", "g"),
                ("u", "N·m", "r"), ("bias", "deg/s", "m")]):
            pw = self.plot.addPlot(row=row, col=0, title=f"{name} ({unit})")
            self.curves[name] = pw.plot(pen=pg.mkPen(color, width=1))
            self.data[name] = ([], [])
        self.vref = QtWidgets.QDoubleSpinBox(); self.vref.setRange(-0.5, 0.5)
        self.vref.setSingleStep(0.05)
        btn = QtWidgets.QPushButton("下发 v_ref")
        btn.clicked.connect(self.send_params)
        for w in (QtWidgets.QLabel("v_ref (m/s)"), self.vref, btn):
            panel.addWidget(w)
        self.t = SerialThread("COM5")       # 按实际端口修改
        self.t.frame_ready.connect(self.on_frame)
        self.t.start()

    def on_frame(self, fr):
        th, thd, v, u, bias = fr
        self.idx += 1
        for key, val in (("theta", th), ("v", v), ("u", u), ("bias", bias)):
            xs, ys = self.data[key]
            xs.append(self.idx); ys.append(val)
            if len(xs) > 2000:              # 滚动窗口约 20s
                xs.pop(0); ys.pop(0)
            self.curves[key].setData(xs, ys)

    def send_params(self):
        payload = struct.pack("<f", self.vref.value())
        head = bytes([0xAA, 0x55, 0x02, len(payload), 0])
        crc = crc16(head[2:] + payload)
        self.t.ser.write(head + payload + bytes([crc & 0xFF, crc >> 8]))

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    win = MainWindow(); win.show()
    sys.exit(app.exec_())
```

6.4 无车联调：先打通上位机，再碰实车

调参工具链可以在没有整车的情况下验证：写一个 30 行的“虚拟小车”脚本，用第 4 章的闭环模型按 5ms 步长生成 (θ, v, u)，再按清单 5-7 的帧格式打包，经虚拟串口对（Windows 用 com0com，Linux 用 pty）发给上位机。曲线形态应与图 4-1 的仿真一致。价值：上位机的解析、绘图、下发三块与固件解耦开发，实车联调时唯一的新变量只剩硬件本身；同时虚拟小车也是参数帧协议的回归测试环境。

> **大白话类比** 虚拟串口联调相当于飞行模拟器：仪表盘（上位机）先和模拟舱（模型）磨合好，真机试飞时你的注意力全放在飞机上，而不是仪表上。

第 7 章　测试验证与结果分析

7.1 测试平台与仪器

平面硬化地面（瓷砖/水泥，避免地毯——摩擦特性完全不同）；ST-Link 烧录器；USB 转 TTL 接上位机；平地尺/量角器（中值标定辅助）；秒表与手机慢动作（恢复时间粗测，精确值以遥测曲线为准）。所有“实测值”的口径：以上位机记录的 CSV 数据统计为准，肉眼判断只做初筛。

7.2 测试用例与记录表

表 7-1 测试用例矩阵（实测值在第 6.3 节界面或 CSV 中量取）

编号	用例	方法	合格判据	实测值
T1	静态平衡波动	立正静止记录 60s 遥测	θ 峰峰 ≤±1.0°	【待填】
T2	抗冲击恢复	侧向轻推至约 10° 摆角释放	2s 内 θ 回 ±2° 内	【待填】
T3	速度阶跃跟踪	下发 v_ref=0.3m/s	稳态误差 ≤0.02m/s	【待填】
T4	速度回零漂移	0.3m/s 跑 10m 后归零	停车点漂移 ≤0.15m	【待填】
T5	上电自站立	扶正释放 20 次统计	成功率 ≥90%	【待填】
T6	缓坡工况	3° 斜坡匀速通过	不发散，速度误差 ≤0.05m/s	【待填】
T7	长跑稳定性	连续运行 30min	无重启、无丢帧（SEQ 连续）	【待填】

7.3 标定与整定流程（按序执行，一步没绿不进下一步）

第一步：符号自检仪式。闭环前用手做三个动作，在上位机曲线上确认：手抬车头向前倾，θ 必须变正；手向前推车，v 必须变正；手动给正占空比，轮必须向前。任一不对就改符号定义，三个全对才允许闭环——符号错一个等于正反馈，当场飞车。

第二步：零偏标定。车静止躺平上电，固件自动采 400 样本。验证：静止时遥测 θ 曲线漂移 <0.1°/min。

第三步：死区测量。直立环关闭，占空比从 1% 逐步加大到轮子刚开始转，记录该值即 DEADZONE（样车 6%）。补过头（补偿量大于实际死区）会表现为小幅度高频抖，此时把补偿系数回退 20%。

第四步：机械中值标定。全环开、平地上运行：若车持续朝一个方向匀加速跑，把中值角往那个方向修。判据：长时间平均速度为零。原理：中值偏 0.5° 等效 0.086m/s² 的恒定加速度，积分项只能补误差不能补物理。

第五步：直立环整定。只动 Q_1 与 R_ctrl 两个旋钮，方向按表 4-2。判据：T1 与 T2 达标。

第六步：速度环整定。先 P 后 I：Kp 从 0.5 起加到推车能明显感到回拉；Ki 加到缓坡能稳住为止。判据：T3、T4 达标。若加 I 后出现几十秒周期的“呼吸”摆动，是积分过强，减半。

第七步：滤波微调。静态 θ 曲线毛刺大→R 减小一档；出现低频慢摆（醉酒晃）→R 增大或 DLPF 回调。每改一次滤波重跑 T1、T2。

7.4 典型故障排查手册

表 7-2 常见故障与定位方法（症状 → 根因 → 解法）

现象	最可能原因	定位手段 / 解法
上电即朝固定方向猛冲	θ 或 θ̇ 符号接反（正反馈）	7.3 第一步符号自检；改 atan2 负号或 gyro 取反
总朝一边匀加速跑	机械中值偏差	0.5° 偏差=0.086m/s² 等效加速度；按 7.3 第四步标定
小幅等幅振荡不停	电机死区 + 积分爬行形成极限环	实测死区并补偿；检查积分限幅
速度环一开整车高频抖	编码器量化噪声（4.5.3 节 8%）+ 低通不足	拉长速度窗、调轻低通、速度环降频
低频大角度慢摆（醉酒晃）	滤波链相位滞后吃掉裕度	R 调小、DLPF 从 5Hz 档退回 42Hz；重跑 T2
跑几分钟 MPU 数据断流	I2C 死锁（SDA 被从机拉低）	9 个 SCL 脉冲恢复（清单 5-2）；I2C 布线远离电机线

现象	最可能原因	定位手段 / 解法
控制周期忽长忽短	控制算在主循环 / 中断被抢占	GPIO+示波器实测 jitter；控制放 TIM6 最高优先级
满电正常，越跑越软	电池电压跌落，等效出力缩水	ADC 采 VBAT，输出乘 VBAT_NOM/VBAT 归一化
上电以歪姿态起飞	启动时卡尔曼初值错 / 车未扶正	启动 gating（清单 5-8）：静止+竖直 200ms 才闭环
换地板就失灵	摩擦特性改变 → 死区改变	死区与中值是环境参数，换环境重标（7.3 第三、四步）

答辩高频点 面试官最爱问“调车遇到过什么问题”。讲一个完整归因链：现象（车总朝右匀加速跑）→ 第一反应（以为是速度环参数）→ 数据归因（速度均值不为零但 θ 均值为 0.4°）→ 物理本质（中值偏差=重力分量，积分补不了物理）→ 解法（标中值）→ 复测（T4 通过）。有数据、有机理、有验证的调试故事比任何创新点表述都加分。

第 8 章　技术栈总表与简历改写

8.1 技术栈分层全景

表 8-1 技术栈分层清单（面试按此表自顶向下讲）

层	技术项	掌握程度自评口径
建模	牛顿-欧拉受力分析、小角度线性化、能控性判据	能徒手推导 (A)(B) 两方程并解释每个系数符号
估计	最小方差估计、二维卡尔曼五公式、零偏在线估计	能从 $K=P^{-}/(P^{-}+R)$ 一路推到代码每一行
控制	变分法、Riccati 方程、LQR、LQI 增广、鲁棒裕度	能完整推导并解释 Q/R 的物理意义
MCU 驱动	TIM/PWM/I2C/编码器接口/ADC/NVIC/IWDG/DMA	寄存器级理解，非只会 CubeMX 点选
算法实现	定点/浮点权衡、度-弧度一致性、限幅与抗饱和、死区补偿	每个工程细节能说出为什么
实时系统	中断调度、时序预算、jitter 度量、故障安全	能讲清每一毫秒的去向
上位机	pyserial、QThread+信号槽、pyqtgraph、流式帧解析	可长期无人值守运行并在线调参
工具链	Keil MDK/GCC、ST-Link、Git、Octave/MATLAB、numpy 仿真	设计-仿真-实现-验证闭环全流程

8.2 一分钟项目介绍（面试/答辩开场通用）

话术模板　我做了一个双轮自平衡机器人，核心是把“估计—建模—控制”整条链路自己推了一遍。传感器端，MPU6050 的加速度计动态不可信、陀螺仪有零偏漂移，我用最小方差准则推导了一个二维卡尔曼滤波器把两者融合，状态取倾角和陀螺零偏，零偏可以在线学出来。被控对象上，我用牛顿-欧拉法建立了轮式倒立摆的四状态模型，机械参数用摆线法和两工况法实测，模型给出开环不稳定极点约 15.7rad/s，从 1° 倒到 10° 只要 191 毫秒，这决定了 5 毫秒控制周期的选择。控制器用 LQR：从哈密顿函数推到黎卡提代数方程，离线解出全部增益，板上只做四次乘加；速度环利用“加减速必须先倾斜”的物理机理做成串级结构，速度误差输出目标角偏置。调参不是试凑：Q、R 每个权重都

> 有物理含义和整定方向，还做了仿真先行和无车联调。最终静态平衡波动小于 1 度，侧推 10 度两秒内恢复。

8.3 简历项目经历改写（对照版）

改写原则：每条都带【方法 + 量化结果】，把能被追问的技术点主动暴露出来，逼自己准备到能讲透。

表 8-2 简历写法对照

原表述	升级表述
使用 MPU6050 获取小车姿态角度	实现寄存器级 MPU6050 驱动与二维卡尔曼姿态融合（状态 [θ, 零偏]），静止 400 样本零偏标定 + 在线自学习，静态角度波动 ≤±1.0°
用 PID 控制小车平衡	按牛顿-欧拉法建立轮式倒立摆四状态模型（参数摆线法/两工况法实测），LQR 最优控制（Riccati 推导+离线求解），静态波动与抗扰恢复均优于串级 PID 基线
实现小车速度控制	设计串级速度闭环：速度 PI 输出目标角偏置（先仰后动机理），含抗积分饱和与死区补偿，速度阶跃稳态误差 ≤0.02m/s
开发上位机软件进行调试	开发 PyQt5+pyqtgraph 四通道遥测调参上位机（自定义 CRC16 帧协议，921600bps），支持参数在线下发与对抗性帧解析
解决了小车不稳定的问题	用摆线法实测转动惯量把模型误差压到 5% 内；建立“符号自检-零偏-死区-中值”四步标定流程，上电自站立成功率从 40% 提升到 90%+

第 9 章　开发计划与里程碑（10 周）

表 9-1 十周开发计划

周次	任务	里程碑产出
W1	理论攻坚：2.2~2.5 传感机理与滤波推导；精读倒立摆建模文献	推导笔记 + 开题报告初稿
W2	第 4 章建模与清单 4-1/4-2：参数表、LQR 设计、闭环仿真	K 增益 + 图 4-1 级仿真曲线
W3	机械组装与第 3.4 节参数实测（吊线/摆线/两工况）	实测参数表回填模型
W4	MPU6050 驱动 + 零偏标定 + 卡尔曼上板	遥测 θ 曲线与 acc 角对比吻合
W5	编码器测速 + 死区测量 + 电机驱动	开环推车运动正常，遥测四通道符号全对
W6	直立环闭环（先扶车后放手）+ 中值标定	自由站立 ≥30s
W7	速度环整定 + 抗扰测试	表 7-1 的 T1~T4 达标
W8	上位机联调 + 在线调参 + LQI 对比实验（论文素材）	全链路界面运行 + 对比数据
W9	测试矩阵全跑 + 长跑稳定性 + 录像	表 7-1 全部回填 + 演示视频
W10	论文撰写、图表整理、答辩演练（第 11 章自测）	论文初稿 + 答辩 PPT

缓冲建议：W1 的理论推导与 W3 的机械加工可并行；W6 第一次闭环失败是常态（第 7.4 节的十个症状里大概率能对号入座一个），预留的时间不要提前花掉。无车联调（6.4 节）是时间紧张时最重要的保底手段。

第 10 章　论文与开题写作映射

10.1 论文章节结构建议（六章制）

表 10-1 论文章节与本文档素材对应

论文章节	内容	可直接取用的素材
第 1 章 绪论	移动机器人与倒立摆背景、国内外研究现状、论文结构	1.3 节对标表、4.7 节 PID 对照
第 2 章 系统方案	总体架构、指标体系、器件选型	图 1-1、表 1-1、表 1-2、表 3-1
第 3 章 硬件与参数	电路设计计算、误差预算、参数实测	3.2/3.3/3.4 节、表 3-2
第 4 章 建模与控制器设计	动力学推导、LQR/Riccati、卡尔曼推导、离散化	4.1~4.5 节全部推导与清单 4-1
第 5 章 软件实现	固件架构、时序、上位机	图 5-1、清单 5-1~6-2 的讲解文字
第 6 章 实验与分析	测试矩阵、标定流程、PID 对照实验、结论	表 7-1 实测数据、7.3/7.4 节、LQI 对比数据

10.2 图表清单

图：系统架构图（图 1-1）、开环发散与周期选择（图 2-1）、卡尔曼融合效果（图 2-2）、LQR 闭环仿真（图 4-1）、时序预算（图 5-1）、上位机界面截图、T1~T3 遥测实测曲线、LQR 与 PID 对照曲线。表：指标表（表 1-2）、误差预算（表 3-2）、参数实测表（表 4-1）、测试矩阵（表 7-1）。全部图表均有脚本或实测数据可复现，避免答辩被问“图哪来的”。

10.3 参考文献基础列表

- 绪方胜彦. 现代控制工程（第五版）[M]. 电子工业出版社.（状态空间、能控性、线性化章节）

- Åström K J, Murray R M. Feedback Systems: An Introduction for Scientists and Engineers[M]. Princeton University Press.（LQR、状态反馈、鲁棒性）

- Kalman R E. A New Approach to Linear Filtering and Prediction Problems[J]. Journal of Basic Engineering, 1960, 82(1): 35-45.（卡尔曼滤波原始文献，绪论引用）

- Ogata K. 离散时间控制系统[M]. 机械工业出版社.（ZOH 离散化、离散 Riccati）

- Grasser F, D'Arrigo A, Colombi S, Rufer A. JOE: A Mobile, Inverted Pendulum?[J]. IEEE Trans. on Industrial Electronics, 2002, 49(1): 107-114.（轮式倒立摆经典系统论文）

- InvenSense. MPU-6000/MPU-6050 Product Specification, PS-MPU-6000A-00[S].（寄存器与噪声规格）

- ST. RM0090 Reference Manual: STM32F405/407[EB/OL]. st.com.（TIM 编码器模式、I2C 时序、NVIC）

- Toshiba. TB6612FNG Datasheet[S].（驱动能力与死区规格）

- Franklin G F, Powell J D, Workman M L. Digital Control of Dynamic Systems[M]. Tsinghua University Press.（采样周期选择、数字实现）

第 11 章　面试与答辩手册（40 问）

每题先给标准答案要点，再给加分句。建议先自测：遮住答案能完整讲出 35 题以上再上答辩场。

11.1 传感器与滤波类（10 问）

Q1 加速度计和陀螺仪各测什么？为什么单用任何一个都不行？

加速度计测比力，静止时等于反重力方向，用 atan2 反正切得角度；陀螺仪测角速度，积分得角度。单用加速度计：车一动，运动加速度直接混进读数，0.98m/s² 就造成约 5.7° 的假倾角；单用陀螺仪：零偏 0.5°/s 一分钟漂 30°，且永远不会自己回来。两者缺陷互补，必须融合。

加分句：加速度计长期绝对可信、动态完全不可信；陀螺仪短期极准、长期不可信——滤波就是按这个互补结构分配信任。

Q2 卡尔曼状态为什么取 [θ, 零偏] 而不是 [θ, θ̇]？

θ̇ 陀螺仪直接测得到，不需要估计；零偏才是真正未知且慢变的量。若状态取 [θ, θ̇] 且把陀螺当输入，两个状态全部由同一输入驱动、观测只含 θ，θ̇ 通道不可观，滤波器退化。

加分句：取 [θ, b] 还附带收益：K_2 通道把持续同号的观测残差自动吸进零偏估计，等于免费做了零偏在线标定。

Q3 卡尔曼增益 K 的物理意义是什么？怎么推导的？

一维下令估计误差方差 $P=(1-K)^2P^- + K^2R$ 对 K 求导取零，得 $K=P^-/(P^-+R)$：预测方差越大越信观测，观测噪声越大越信预测。信任比例由方差之比唯一决定，这是最小方差意义下的最优分配。

Q4 Q 和 R 怎么定？

R 实测：车静止采 5000 点加速度计角度算方差，样车 $0.03°^2$。Q_angle 是陀螺积分每步的失配方差（量级 1e-3），Q_bias 是零偏随机游走强度（量级 3e-3）。整定方向：R 调大更平滑但滞后大，车会低频慢摆；R 调小响应快但振动噪声进 θ。

加分句：R 不是拍的——它是一个可以现场重测的物理量，传感器换一颗、车换一台都应重标。

Q5 卡尔曼和互补滤波什么关系？

互补滤波 angle=k·(angle+gyro·dt)+(1−k)·acc_angle 里那个 k，就是卡尔曼增益收敛到稳态后的值。互补滤波是稳态化简化版；卡尔曼自动算出最优 k，还多出零偏估计能力。两者计算成本都极低，没有理由不用卡尔曼。

Q6 为什么 DLPF 选 42Hz 而不是更低的 5Hz？

DLPF 越低噪声越小，但相位滞后越大。滤波滞后直接从 LQR 的 60° 相位裕度里扣，5Hz 档的滞后在 10rad/s 处已经很可观，会让整车出现低频慢摆。42Hz 先压掉电机机械振动的高频段，剩余噪声交给卡尔曼按方差处理。

加分句：每次加滤波都在用裕度换噪声——这个权衡意识比具体数值重要。

Q7 上电零偏标定为什么必须静止？

标定是把陀螺读数均值当零偏。若标定期间车在动，真实角速度就被吸进零偏，之后所有积分全错，且卡尔曼需要很长时间才能把这个错误零偏洗回来。所以固件 gating 要求静止才标定、静止才启动。

加分句：更讲究的做法是标定后检查样本方差，方差过大说明没静止，自动重采。

Q8 为什么 MPU6050 数据要一次事务突发读？

14 字节（加速度 XYZ、温度、陀螺 XYZ）在一条 I2C 事务里连续读出，保证属于同一采样时刻。分两次读，两次之间可能跨越传感器内部寄存器更新，得到“加速度是 t、陀螺是 t+1”的撕裂数据，卡尔曼的预测-观测配对就错了。

Q9 I2C 总线死锁是什么？怎么处理？

主机在读中途被复位后，从机可能停在“应答中拉低 SDA”状态，总线永久卡死。恢复方法：把 SCL 切成 GPIO 输出，手动打 9 个时钟脉冲让从机释放 SDA，再发 STOP，最后重新初始化外设。工程上还要配读超时，失败立即断电机力矩。

加分句：这一条属于“长期运行可靠性”设计：不依赖最坏情况不发生，而是发生时行为已知。

Q10 滤波滞后对整车的影响怎么量化？

LQR 的鲁棒性保证建立在全状态反馈成立的前提上，滤波是全链路里吃相位的环节。经验处置：θ 输出滞后控制在 20ms 量级以内（R 的选择决定），速度通道低通滞后约 20ms（4.5.3 节按量化噪声反推）。任何一次改滤波参数，都要重跑 T1/T2 测试验证整车。

加分句：把“滤波延迟 × 不稳定极点 15.7rad/s ≈ 相位损失”这个估算讲出来，能直接体现对闭环的理解。

11.2 建模与动力学类（8 问）

Q11 你的车为什么能倒？开环极点是多少？

小角度下 $\ddot{\theta} = 245.8 \cdot \theta$，系数为正的角度正反馈。开环极点 ±15.7rad/s 与 ±j4.8。物理：重力对轮轴的力矩随倾角增大而增大。15.7rad/s 折算成时间尺度：1° 涨到 10° 只需 191ms，这是全系统时间预算的来源。

加分句：报数字时同时报参数（M=1.5kg、L=0.1m、I_b=0.02），说明数字是算的不是背的。

Q12 建模做了哪些假设？什么时候失效？

三条：小角度（$\sin\theta \approx \theta$，$\cos\theta \approx 1$）、轮子纯滚动不打滑、忽略电机反电动势动态。失效场景：大角度（>15° 后 $\cos\theta$ 项不可忽略，但那时平衡已失败，失效区不在工作区）；地面打滑；高速时反电动势削力矩（已由电压前馈缓解）。

加分句：加一句“失效区避开工作区，是线性化建模合法性的标准论证”。

Q13 为什么车前倾时必须向前开？

前倾 θ>0 时重力矩继续把车往前摔，唯一反向力矩来自车轴加速产生的惯性力：$b_1 = -2ML/(rD) < 0$ 说明正力矩（向前加速）产生负的角度加速度——车向前“钻”到质心下面把倒下的趋势接住。人接竹竿同手同脚往前跑是同一个物理。

加分句：这就是 K_1 必须为负的原因（$u = -K \cdot x$），符号推导与物理图像互为印证。

Q14 M_eq、I_b、D 各是什么？

$I_b=J_p+M_pL^2$ 是车体绕轮轴的转动惯量（平行轴定理）；$M_{eq}=M_p+2(J_w+mr^2)/r^2$ 是把车轮折算进平动的等效质量；$D=I_b\cdot M_{eq}-M_p^2L^2$ 是消元后的判别式，必须大于零，否则两个二阶方程线性相关、模型退化。样车 D=0.0096。

Q15 车轮的动量能忽略吗？

不能。忽略车轮会把 a_2、b_2 全部丢掉——速度通道的动态就错了，而速度闭环恰恰依赖这条通道。定量：车轮折算质量 $2(J_w+mr^2)/r^2\approx0.105$kg，占 M_eq 的 6.5%，小但系统性影响速度响应；更关键的是 $a_2=-23$ 那一项描述“车倒下时轴后退”的反冲，是先仰后动现象的模型基础。

Q16 模型参数怎么测的？

质心高度 L 用吊线法（两次悬挂铅垂线交点）；绕轴惯量 I_b 用摆线法（车体绕轴自由微摆，$I_b=M_pgLT_p^2/4\pi^2$）；电机 Ke、R_m 用两工况稳态法解 $U=Ke\cdot\omega+I\cdot R_m$，SI 制下 Kt=Ke。全部半天内完成，测量误差约 5%。

加分句：摆线法本质是用重力恢复力标定惯量，比“细杆近似”把模型误差从 30% 压到 5%，直接决定 K 的准确度。

Q17 能控性怎么验证？验证它有什么用？

构造可控性矩阵 $[B\ AB\ A^2B\ A^3B]$，秩为 4 则完全可控。用处是理论通行证：系统可控才存在任意极点配置和有界最优 LQR 解；若某个状态（比如位置）不可控，LQR 问题无解或必须降阶处理。

加分句：顺手说一句：本设计四状态全可测（卡尔曼+编码器），不需要设计观测器，这是选 LQR 而不是 LQG 的原因。

Q18 模型不准，LQR 还灵吗？

灵，这就是选 LQR 的理由之一：全状态反馈 LQR 有 ≥60° 相位裕度、≥6dB 增益裕度的理论下界，5% 的参数误差远不足以吃掉它。系统性偏差（如死区、坡度）表现为常值扰动，由速度环积分消除。模型误差的代价不是失稳，而是“最优性打折”——恢复轨迹次优。

加分句：诚实补充：传感器坏掉或滤波发散时保证失效，所以故障安全（I2C 失败断力矩）是必需品。

11.3 LQR 与控制类（10 问）

Q19 LQR 和 PID 的本质区别？

PID 逐环试凑，环间耦合靠经验；LQR 把多变量耦合写进 A、B 矩阵，从 Q/R 权重一次解出全部增益，附最优性与鲁棒裕度保证。两者联系：直立环 PD 恰好等于 LQR 增益前两项，PID 可以理解为 LQR 解的手工低配版。

加分句：答辩 safest 表述：PID 的优势是不需要模型；我有模型且参数可实测，所以选 LQR——这是方法选择逻辑，不是口味。

Q20 Q、R 怎么选？调它们像调什么？

Q 是状态误差的罚款单价，R 是控制量的油钱单价。方向表：Q_1↑ 抗扰强但高频抖；Q_2↑ 阻尼强；Q_4↑ 抗推；R↑ 温柔省电但软。起点 diag(50,2,1,100)/R=1，仿真确认后上车微调。调 Q/R 与调 PID 旋钮的区别：改的是“价目表”，K 由理论自动重算，各增益之间的比例关系始终自洽。

Q21 为什么 K 有负数？

符号约定：θ 前倾为正，需要的力矩也向前为正，而公式是 $u=-K\cdot x$，所以 K_1 必须为负。数值 K=[−2.798, −0.326, −0.094, −0.971]。把负号“修正”掉是新手第一大坑——正反馈当场飞车。

加分句：补一句验证方法：手推车头，遥测 u 必须同向变大，四个增益的联合符号一次自检。

Q22 为什么 K 在 PC 上离线算，不在 STM32 上解 Riccati？

Riccati 求解是矩阵迭代（Schur 方法或迭代法），四维系统要几百微秒到毫秒级而且解一次终身不变——在线解毫无收益。正确分工：离线算 K，板上每周期只做 4 次乘加，约 1μs。

加分句：这是嵌入式控制的一条通用纪律：能离线的绝不在线，把 CPU 留给采样和通信。

Q23 5ms 采样下连续 K 直接用，为什么可以？

连续 LQR 的 K 用在 ZOH 离散系统上的误差来自极点映射，量级 $(sT)^2/2$：主极点 15.7、T=0.005 时约 0.31%，远小于模型本身的 5% 参数误差。控制周期拉长到 20ms 以上就必须用 c2d 精确离散并重解离散 Riccati。

加分句：给出数字再下结论，比“周期小没问题”高一个层次。

Q24 速度环为什么输出目标角而不是直接给力矩？

轮子的推力只能通过车体倾斜产生：$\ddot{x} \approx g \cdot \theta$。要减速必须先后仰，要加速必须先前倾。所以速度误差的正确出口是目标角偏置 $\theta_t = -k(v - v_{ref})$，直立环再按姿态误差出力矩。直接给力矩的结构会把“加减速”和“保平衡”两个目标搅在同一个信号里打架。

加分句：举人体类比：急刹车身体后仰——平衡车的姿态就是它的油门刹车。

Q25 纯 LQR 有什么问题？怎么消静差？

纯状态反馈是比例控制，对恒定扰动（坡度重力分量、死区）有静差。两条路线：串级结构里速度 PI 的积分项消差（本项目采用，限幅+抗饱和）；或把 $\int (v - v_{ref})$ 升为第五个状态做增广 LQI，积分增益由 Q_5 统一权衡。

加分句：说明为什么 LQI 更“理论”而工程先上 PI：各环独立可调、失败域小是收敛速度优先的工程选择。

Q26 力矩限幅会破坏 LQR 的最优性吗？

会。LQR 的最优性在线性无约束假设下成立，饱和后实际执行的是“饱和后的 LQR”，恢复轨迹由限幅决定。工程处理：把 U_MAX 设在 95% 并核算最强恢复工况（侧推 10°）所需力矩在限幅内；饱和频繁出现说明扰动超出设计包线，该加大 Q_1 或提高电机能力。

加分句：报设计包线的概念：控制器只在包线内承诺性能，包线由仿真确定。

Q27 LQR 的鲁棒性保证条件是什么？什么时候失效？

条件：全状态反馈 + 标准代价函数。保证：每回路 ≥60° 相位裕度、≥6dB 增益裕度。失效场景：状态不可测需要观测器时（相位从观测器里丢）；输出饱和；

滤波滞后过大。本项目四状态全可测，主要威胁就是滤波滞后，所以把它控制在20ms 内。

加分句：把“保证的前提”主动讲出来，比只背“60°/6dB”深刻得多。

Q28 你实际对比过 LQR 和 PID 吗？

做过对照：同一台车，先调串级 PID 立稳（直立 PD+速度 PI），再切换LQR。体感差别：LQR 抗侧推的恢复轨迹更平滑（各增益比例理论自洽，不存在“这个环过强那个环过弱”）；换电池电压、换载重时 PID 要重调，LQR 只需确认限幅。数据以 T1/T2 曲线为准，写进论文第 6 章。

加分句：如果没做完整对照，诚实说“做了两组代表性对比”并给数据范围，不要虚报。

11.4 嵌入式实现类（7 问）

Q29 控制周期为什么 5ms？怎么保证不抖？

选周期的依据：不稳定时间尺度 191ms 的 1/40，且显著快于姿态环带宽。保证不抖的手段：硬件定时器中断、NVIC 最高优先级、中断内无阻塞调用（I2C 带 4ms超时但正常 0.4ms）、遥测走 DMA。验证手段：GPIO 翻转+示波器测 jitter<1%。

加分句：周期抖动对正反馈系统等效于随机延迟注入——这句原理讲出来，“为什么不能用 delay”就不用再解释了。

Q30 中断里能做 I2C 同步读吗？

能但要有界：14 字节@400kHz 约 0.4ms，占周期 8%，可以接受；关键是超时必须设（4ms），失败立即断力矩。更优方案是 I2C+DMA 或分时预取，把中断内时间压到几十微秒——本项目把这条列为改进项。

加分句：答“能、但有界、且我有失败预案”比答“不能”准确。

Q31 看门狗怎么用？喂狗点在哪？

独立看门狗 IWDG 设 500ms，喂狗点在主循环末尾。控制中断若卡死或主循环死机，超时复位，复位后 gating 重新走启动流程，车停而不是乱跑。设计原则：喂狗周期要显著短于超时，且不在中断里喂狗（中断正常不代表主循环正常）。

加分句：补充：复位后第一件事是零力矩——故障安全的默认态是静止。

Q32 float 还是 double？

float。M4F 的 FPU 是单精度硬件（fpv4-sp-d16），double 走软件库慢一个数量级，还会显著增加栈占用。控制与滤波精度需求 1e-6 量级，float 的 7 位有效数字足够。编译必须开 -mfpu 与 hard ABI。

加分句：顺手展示意识：所有角度运算在内部统一单位（弧度或度二选一），换算只发生在边界，避免混用——这是数值 bug 的常见来源。

Q33 编码器测速为什么抖？怎么处理？

量化：5ms 窗口 1 个计数对应 0.024m/s，0.3m/s 目标速度下噪声 8%。处理：一阶低通（系数 0.25，滞后约 20ms）或拉长窗口；代价是相位滞后，不能压太狠。

加分句：补一句口径：抖动的根源是量化不是噪声，换高线数编码器是唯一能同时降噪不增滞后的方案。

Q34 电机死区怎么量化、怎么补偿？

量化：直立环关闭，占空比从 1% 起步进到轮子刚转，记录该值（样车 6%）。补偿：小信号区把输出线性抬到死区之上，d_out = DEADZONE + (1−DEADZONE)·|d|。过补偿症状是小幅高频抖，回退 20%。

加分句：死区补偿只是掩盖，根源治理靠速度环积分——两句一起说才完整。

Q35 为什么要做电池电压归一化？

电机力矩近似正比于 PWM 电压，电池从满电 8.4V 掉到 7.0V 时同样占空比的出力缩水 17%，等效于控制增益漂移——满电偏激进、亏电偏软。ADC 采 VBAT 后给 PWM 乘 VBAT_NOM/VBAT，把执行器增益钉回常数。

加分句：这是“前馈消已知扰动”的最小案例，一句话能带出前馈/反馈分工的理解。

11.5 工程素养与压力问题（5 问）

Q36 和网上开源平衡车相比，你的创新点在哪？

我不声称发明新算法——卡尔曼和 LQR 都是教科书内容。工程价值在组合与闭环：传感误差机理 → 建模参数全部实测 → 控制器从推导到代码逐行对应 → 调参有方法论 → 测试可复现。答辩现场我可以改一个 Q 权重演示整车行为变化，并解释每个参数“为什么是这个值”——这是开源 demo 给不了的。

加分句：真正的差异在细节：符号自检流程、启动 gating、故障安全、量化噪声分析，这些是把 80 分 demo 和毕业设计分开的东西。

Q37 这个项目最难的问题是什么？

讲一个归因链完整的故事：车总朝一边匀加速跑。第一反应是速度环参数问题，调了两天无效；后来用上位机记录发现 θ 均值≈0 但速度均值恒负，意识到这不是参数问题而是物理问题——机械中值偏 0.5°，重力分量就是 0.086m/s² 的恒定加速度积分只能补回路误差、补不了物理偏差。标定中值后 T4 一次通过。收获：先把“参数问题”和“物理问题”区分开，再动手调。

加分句：这个故事同时展示了数据归因能力（不是瞎试）和对物理的尊重（知道积分的边界）。

Q38 怎么证明你的平衡精度是可信的？

三层验证：仿真层，第 4 章脚本的理论轨迹；数据层，上位机 60s 静态记录统计峰峰值；物理层，T2 抗扰与 T5 上电成功率。三层结论一致才算数；任何单层数据（比如只看曲线好看）都不作为指标依据。

加分句：强调口径统一：所有实测值都定义了量取方法（表 7-1），可复现。

Q39 如果重做一遍，你会改什么？

四件事：换 1024 线编码器从根上消量化噪声；MPU 换 ICM-42688（噪声低一个量级）；把增广 LQI 做完整对比实验补进论文；画 PCB 把模拟走线和电机功率分开（现在洞洞板布局限制了抗干扰上限）。

加分句：结尾加一句：这些是性价比排序，不是愿望清单——说明你对工程取舍有概念。

Q40 演示时车突然倒了怎么办？

现场预案三件套：启动 gating 保证“扶正了才启动”；失效检测保证“不对劲就断力矩”（I2C 读失败、角度超限、看门狗复位都落到零力矩）；演示动作全部在第 4.6 节仿真的设计包线内（侧推幅度、坡度），不临场加戏。倒地后按标定流程 30 秒内可重新上电启动。

加分句：评委问这题多半在考察可靠性思维——回答重心放在“失效时行为已知”而不是“我不会倒”。

附录 A 代码文件清单与自检结果

表 A-1 代码文件清单

文件	行数(约)	自检方法
code/design/design_lqr.m	20	rank(ctrb)=4、闭环极点左半平面、与正文 K 值一致
code/design/sim_lqr.py	45	图 4-1 复现：3° 扰动 0.6s 内回零、力矩不饱和
code/stm32/balance_config.h	55	与第 2/4 章理论值逐项对表
code/stm32/mpu6050.c	85	WHO_AM_I 自检；静止 θ 漂移 <0.1°/min
code/stm32/attitude.c	45	与 2.3.3 五公式逐行对表；手持晃动看输出平滑
code/stm32/encoder.c	35	手转轮一圈计数 =1560；速度符号与方向一致
code/stm32/controller.c	40	负 K 校验（4.3 节）；bias 限幅生效
code/stm32/motor.c	55	死区补偿前后低速爬行对比；电压归一化对比
code/stm32/uart_frame.c	40	串口助手十六进制核对帧结构；CRC 抽帧校验
code/stm32/main.c	90	gating 三条件逐条验证；拔 I2C 线验证故障安全
code/host/parser.py	45	垃圾字节+粘包+半包混合灌入回归测试
code/host/gui.py	80	100Hz 刷新不卡顿；参数下发回读生效

附录 B 开源参考与甄别建议

平衡车是开源重灾区，参考可以、照抄危险。甄别三条原则：

- 能跑 ≠ 能讲：很多开源工程参数无出处，答辩被问“Kp 为什么是 8”就露馅。本项目每个参数都能指到理论推导或实测流程；

- DMP 直读角度的工程可以参考驱动写法，但姿态融合部分应替换为本文 2.3 节的自研卡尔曼——DMP 是黑盒且输出 100Hz 固定、延迟不可控；

- 借鉴代码时优先看接口设计与故障处理（符号自检、gating、总线恢复、看门狗），这些才是工程能力所在；核心算法务必自己重写并逐行对应到推导。